

Programmazione Grafica 2d



```
ball.html
<html>
<body>
<script>
  var larghezza=300;
  var lunghezza=300;
  var cg;
  function init() {
    var canvas = document.getElementById('my_Canvas');
    cg = canvas.getContext("2d");
  }
  /*===== Drawing =====*/
  var time_old=0;
  var animate=function(time) {
    cg.clearRect(0,0,larghezza,lunghezza);
    cg.beginPath();
    cg.fillStyle = 'red';
    cg.arc(x, y, r, 0, Math.PI*2, true);
    // cg.closePath();
    cg.fill();
    if(x+r >= larghezza || x-r <= 0) dx = -dx;
    if(y+r >= lunghezza || y-r <= 0) dy = -dy;
    x += dx;
    y += dy;
    window.requestAnimationFrame(animate);
  }
  init();
  animate(0);
</script>
```



Sommario

Abbiamo visto l'ambiente Hardware/Software che permette di fare grafica, i primi elementi su HTML5, canvas e contesto '2d' ed alcuni semplice esempi.

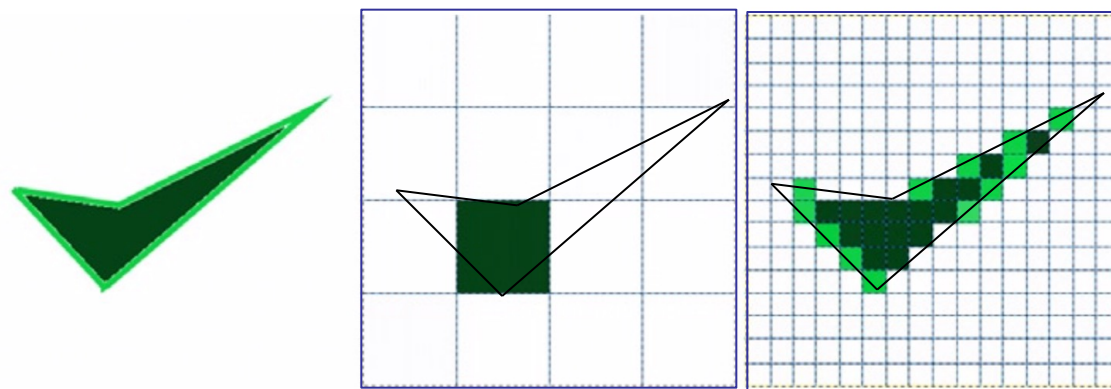
Vogliamo ora introdurre alcuni concetti ed altri esempi particolari di **programmazione grafica 2d**; per la precisione:

1. algoritmo di linea incrementale (per disegno di un segmento/linea in coordinate intere/schermo su **viewport**)
2. algoritmo di Bresenham (per disegno di un segmento/linea su **viewport** usando aritmetica intera)
3. disegno in coordinate floating point (su "**window**")
4. un esercizio da fare insieme
5. un esercizio da fare da soli a casa (**Esercizio 1**)

Immagini e Pixel

Un'immagine digitale è composta da una matrice di punti colorati, noti come **pixel** (immagine raster). Solitamente viene visualizzata su un display. Quanto l'immagine visualizzata appaia fedele all'immagine originale dipende dal numero di pixel del display: la **risoluzione**.

In figura si vede, a sinistra un'immagine, al centro la sua rappresentazione su una griglia 4x4 di pixel e a destra su una griglia 16x16. All'aumentare della risoluzione la differenza fra l'immagine originale e il rendering dell'immagine diminuisce.





Disegno di Punti e Linee

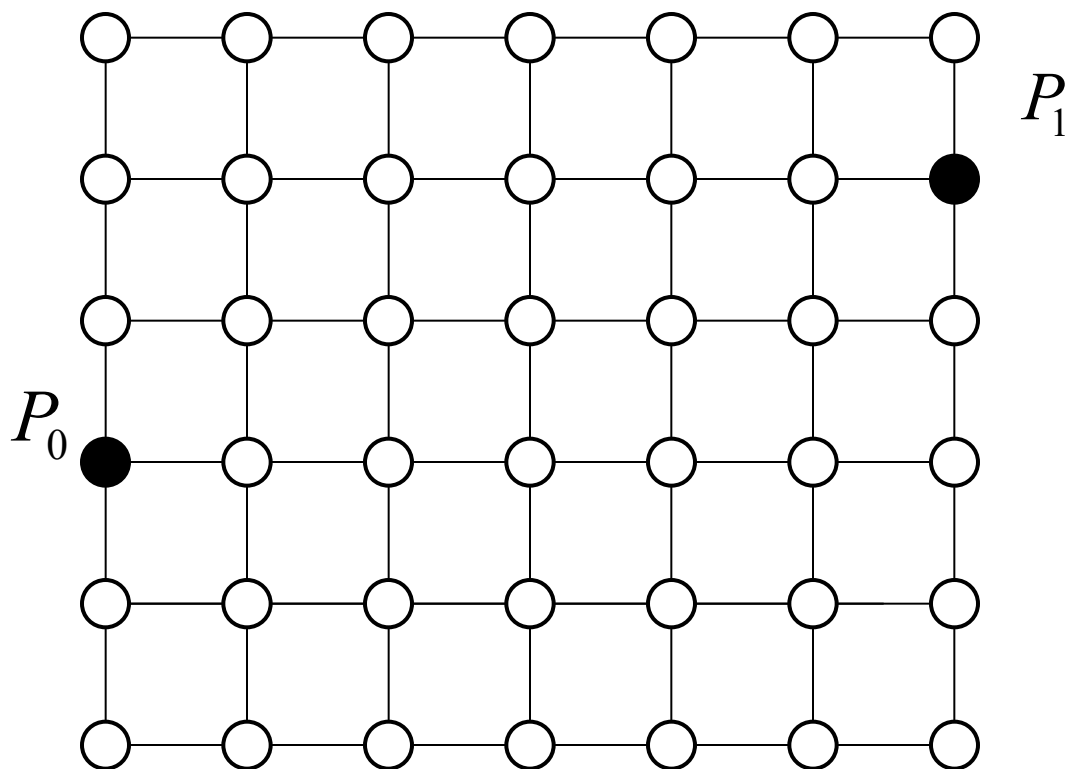
Abbiamo detto che nell nostra API non viene esposta una **primitiva punto**, cioè il disegno di un singolo pixel, mentre esiste la **primitiva linea**;

```
var canvas = document.getElementById("myCanvas");  
var ctx = canvas.getContext("2d");  
  
ctx.moveTo(0, 0);  
ctx.lineTo(200, 100);  
ctx.stroke();
```

Vediamo cosa c'è dietro la **primitiva linea**, cioè al disegno di una linea o segmento retto.

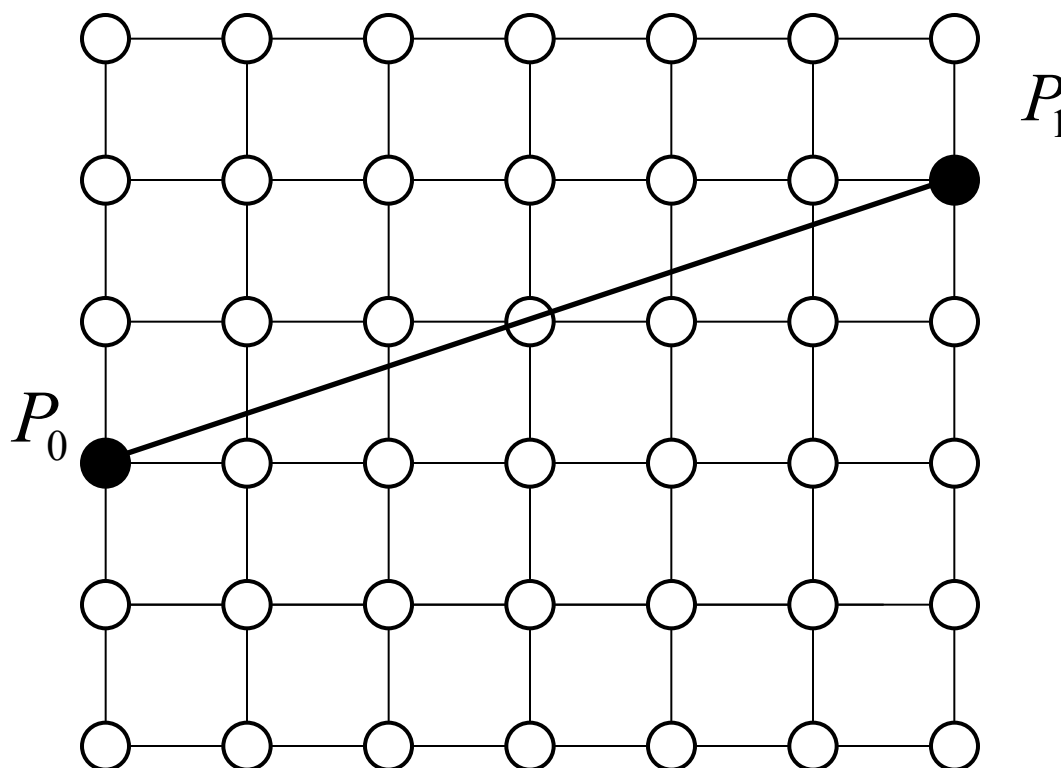
Disegno di Linee

- Dati due punti P_0 e P_1 sullo schermo (a coordinate intere) determinare quali pixel devono essere disegnati per visualizzare la linea (o segmento retto) che definiscono.



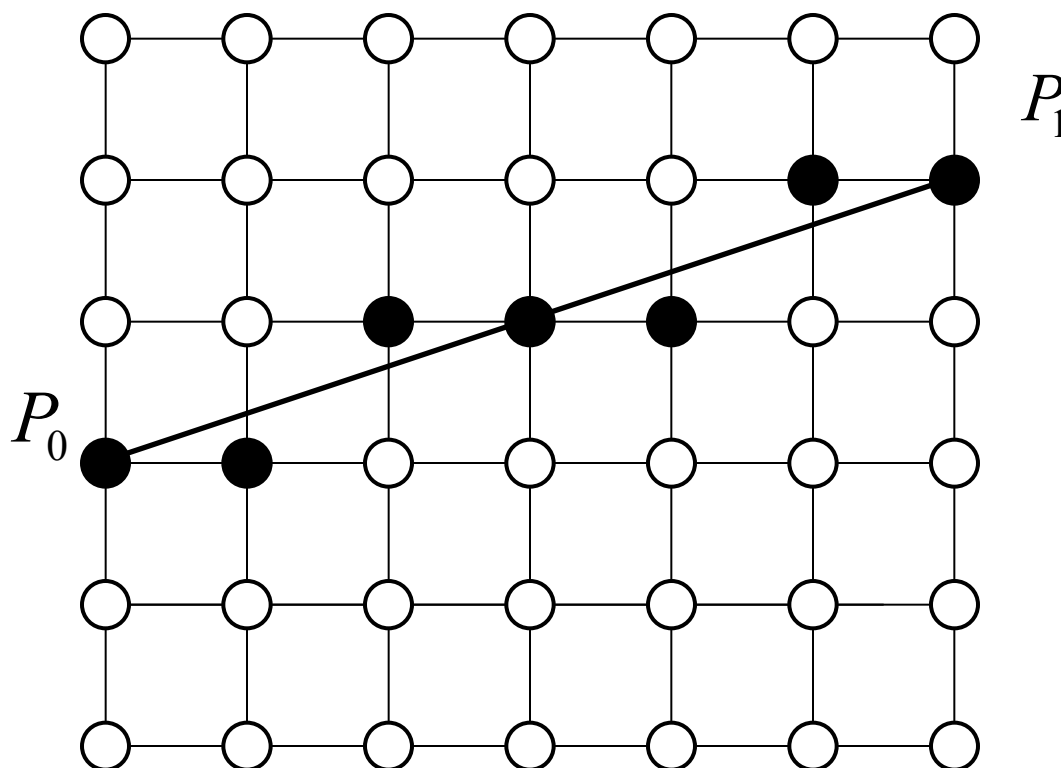
Disegno di Linee

- Dati due punti P_0 e P_1 sullo schermo (a coordinate intere) determinare quali pixel devono essere disegnati per visualizzare la linea (o segmento retto) che definiscono.



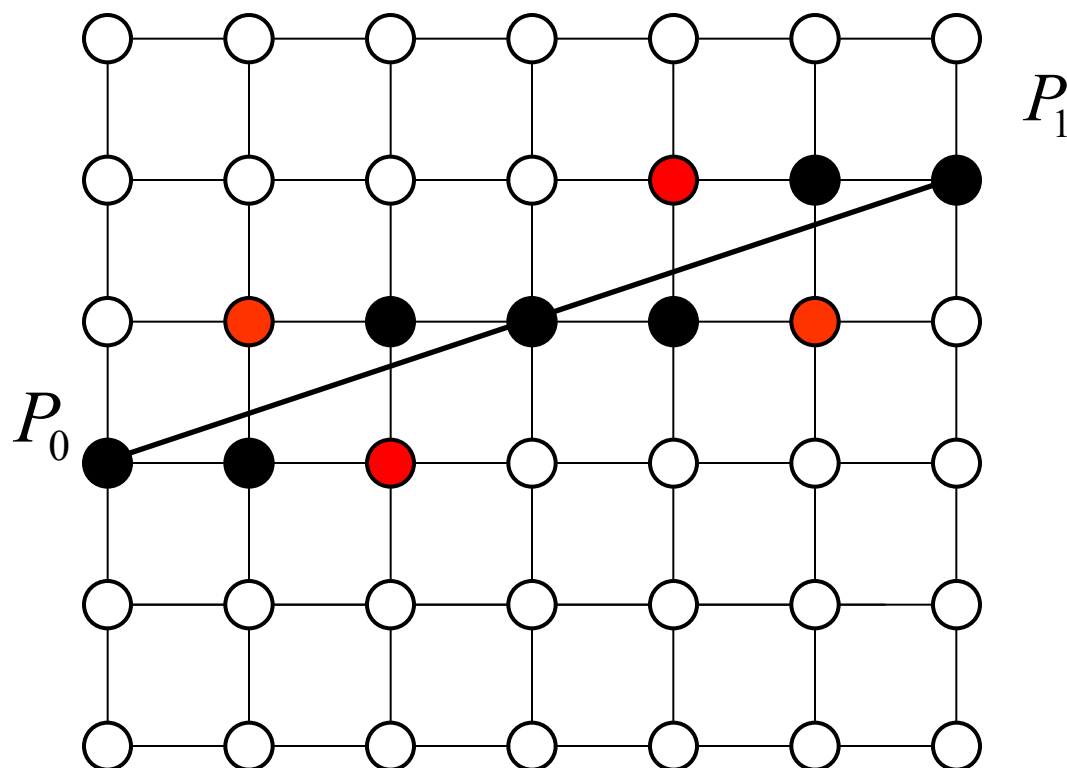
Disegno di Linee

- Dati due punti P_0 e P_1 sullo schermo (a coordinate intere) determinare quali pixel devono essere disegnati per visualizzare la linea (o segmento retto) che definiscono.



Disegno di Linee

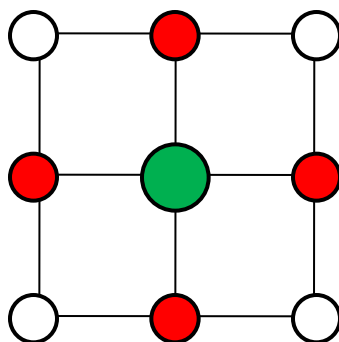
- Perché vengono disegnati questi pixel e non anche altri? come per esempio i pixel in rosso?



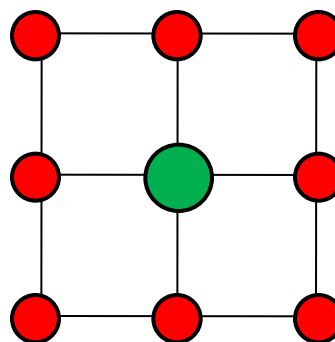
Disegno continuo a Pixel

La logica è di procedere ad “accendere” pixel “adiacenti” per simulare un disegno continuo.

Questo porta alla definizione di pixel adiacenti; ci sono due differenti definizioni:

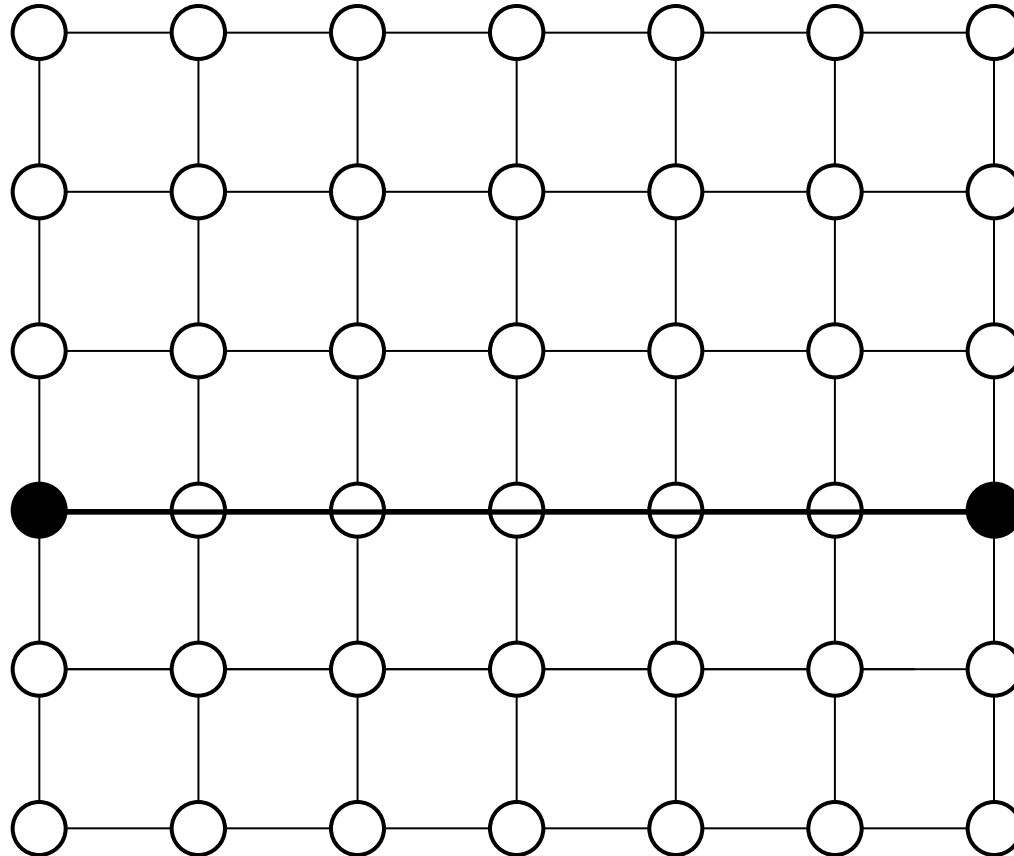


4-connected

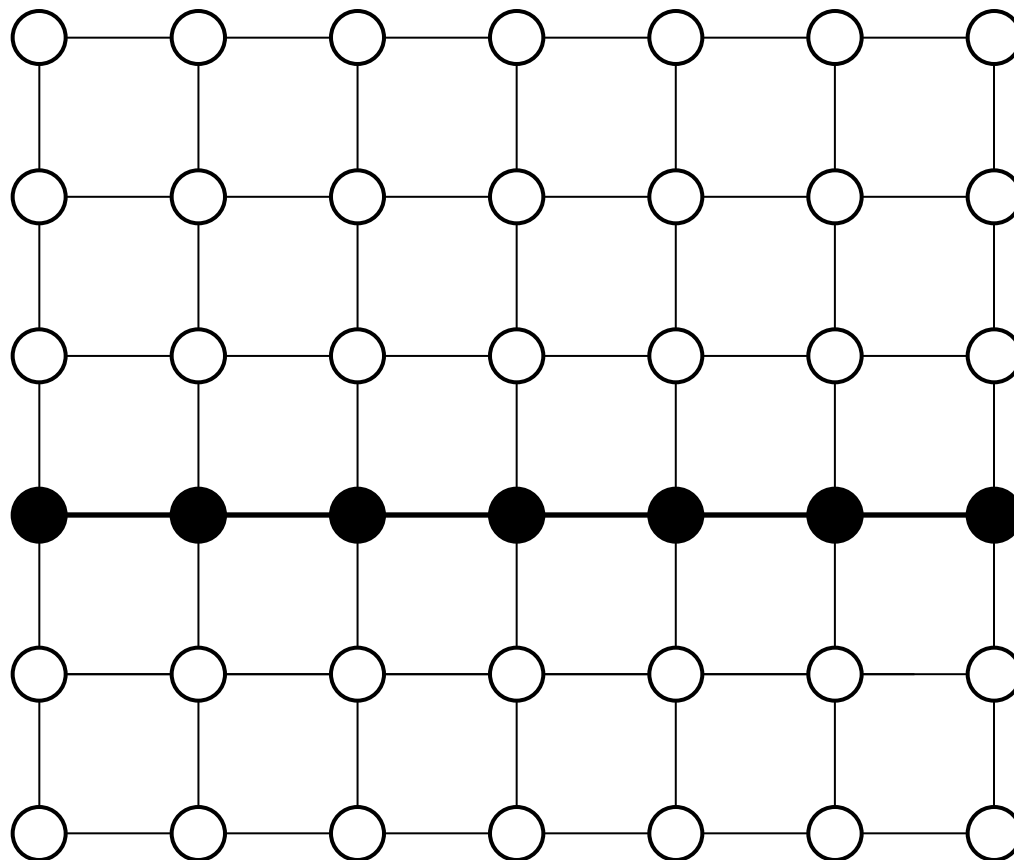


8-connected

Linee Speciali - Orizzontale

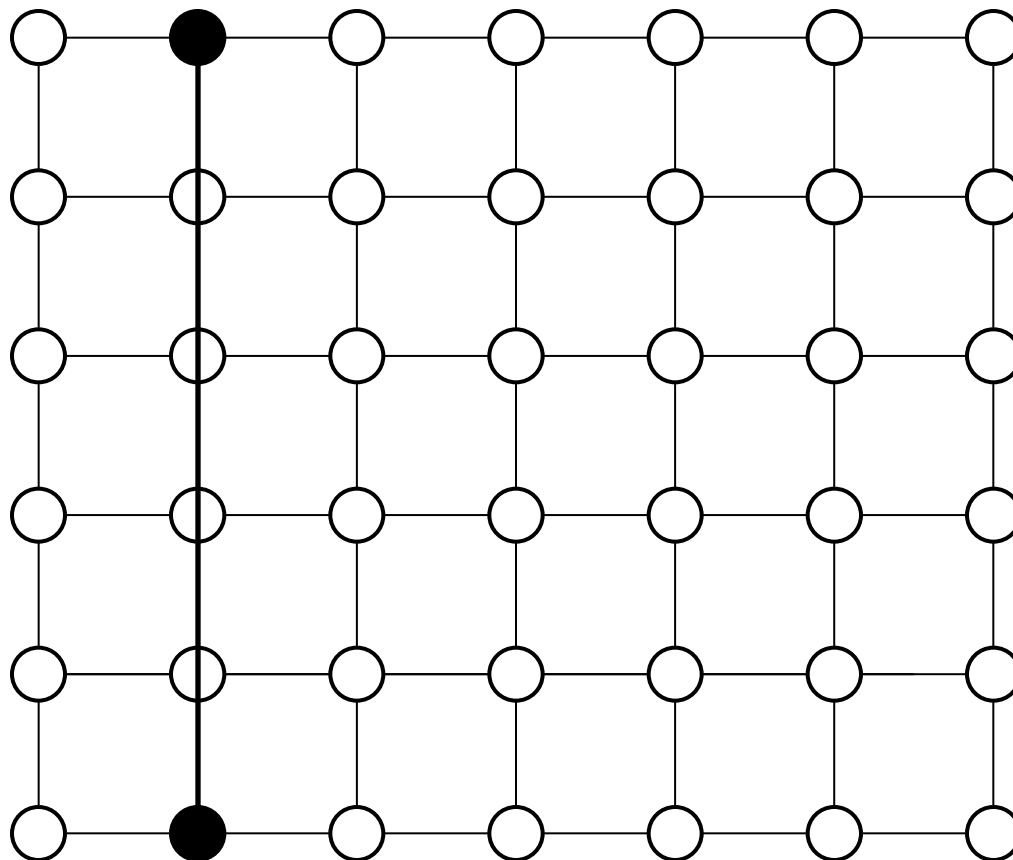


Linee Speciali - Orizzontale

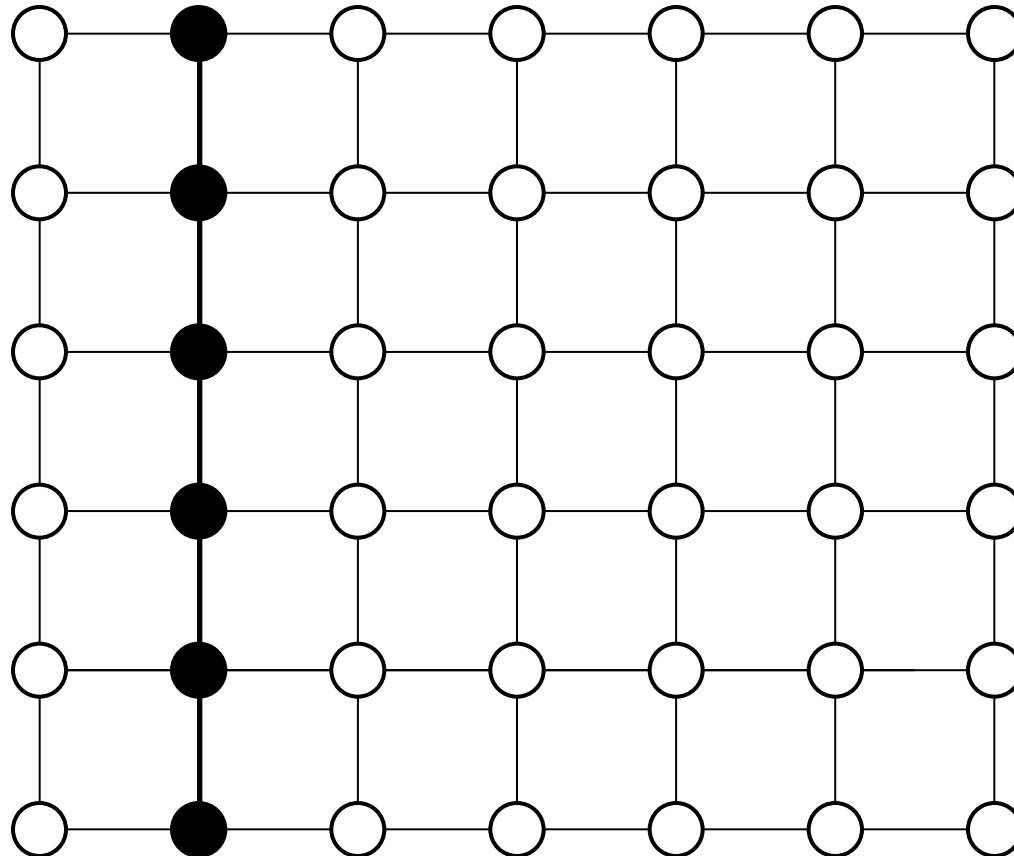


Incrementa la x di 1, tenendo la y costante

Linee Speciali - Verticale

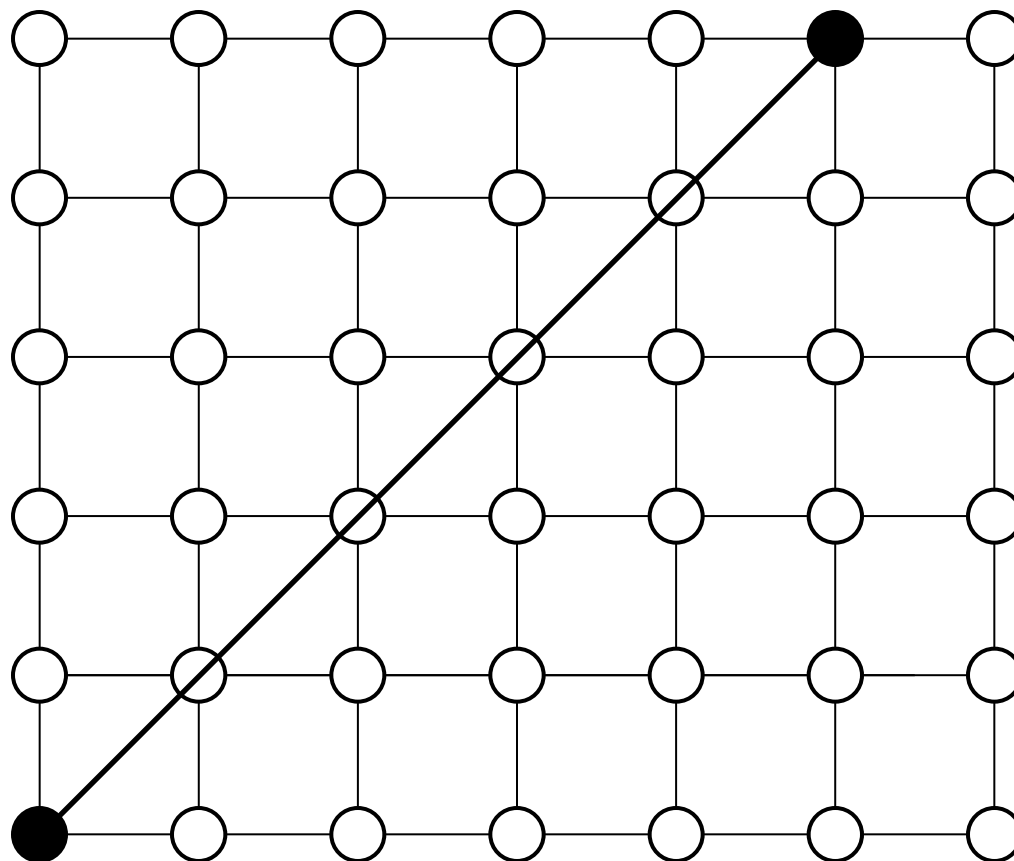


Linee Speciali - Verticale

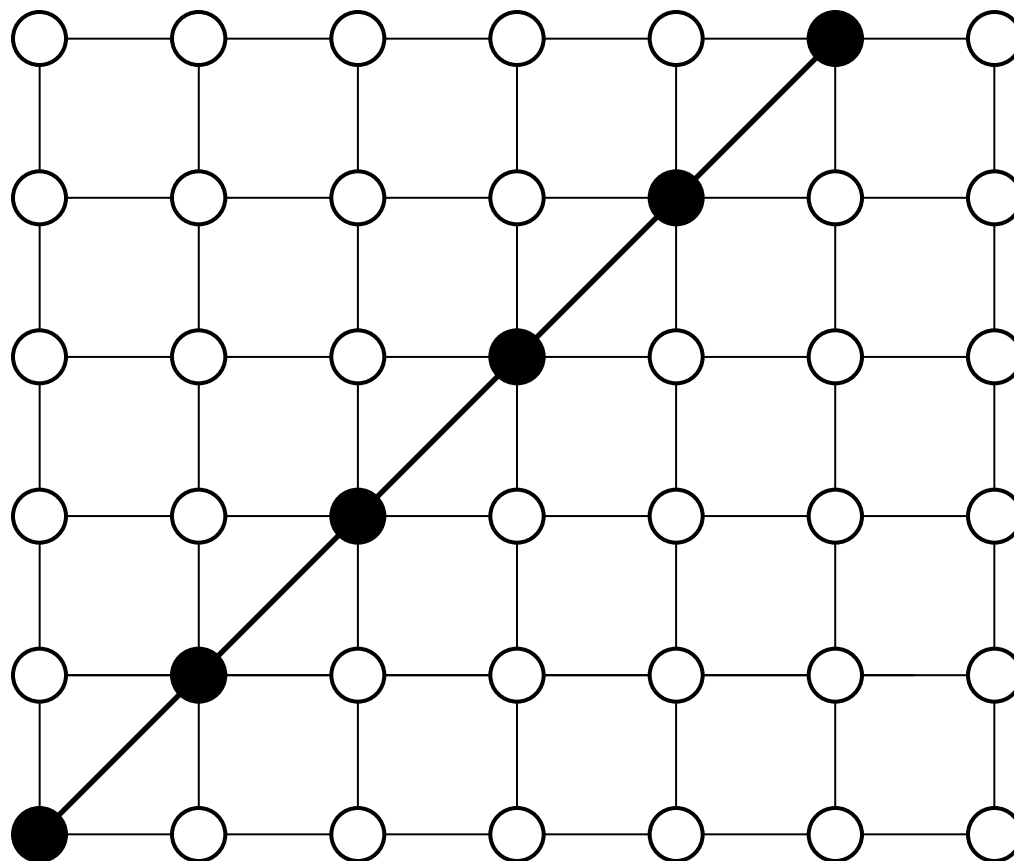


Tieni la x costante e incrementa la y di 1

Linee Speciali - Diagonale

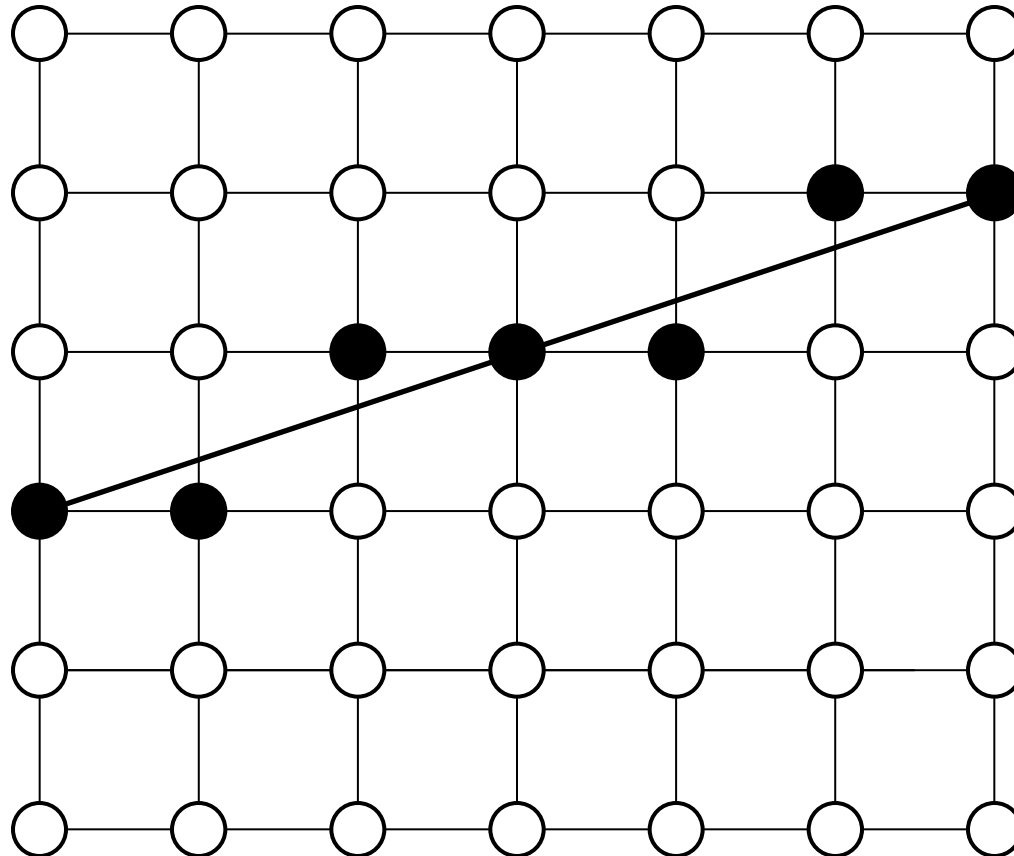


Linee Speciali - Diagonale



Incrementa la x di 1 e incrementa la y di 1

Ma per Linee generiche?





Algoritmo di Linea

Sia $L(t) = P_0 + (P_1 - P_0)t$, con $t \in [0, 1]$, l'espressione in forma parametrica del segmento di estremi $P_0 = (x_0, y_0)$ e $P_1 = (x_1, y_1)$. Dalla forma esplicita

$$\begin{cases} x = x_0 + (x_1 - x_0)t \\ y = y_0 + (y_1 - y_0)t \end{cases} \quad t \in [0, 1]$$

si possono determinare punti del segmento per opportuni valori del parametro t ; il numero di punti è dato dal numero di pixel necessari per rappresentare **8-connected** il segmento.

Algoritmo:

```
n=max(abs(x1-x0),abs(y1-y0))
dx=(x1-x0)/n
dy=(y1-y0)/n
for ( i=0; i<=n; i++) {
    x=x0+i*dx
    y=y0+i*dy
    setPixel(round(x),round(y),col)
}
```

```
int round(float a) {
    int k
    k=(int)(a + 0.5)
    return k
}
```



Algoritmo di Linea Incrementale

Il metodo incrementale consiste nel determinare le coordinate del nuovo punto da quelle del punto precedente, anziché dall'espressione parametrica vista, infatti per le ascisse si ha:

$$x_{i+1} = x_0 + (i + 1) dx$$

$$x_i = x_0 + i dx$$

e sottraendo si ottiene: $x_{i+1} = x_i + dx$; (analogamente $y_{i+1} = y_i + dy$).

Algoritmo:

```
n=max(abs(x1-x0),abs(y1-y0))
dx=(x1-x0)/n
dy=(y1-y0)/n
x=x0
y=y0
setPixel(x,y,col)
for (i=1; i<=n; i++) {
    x=x+dx
    y=y+dy
    setPixel(round(x), round(y), col) }
```

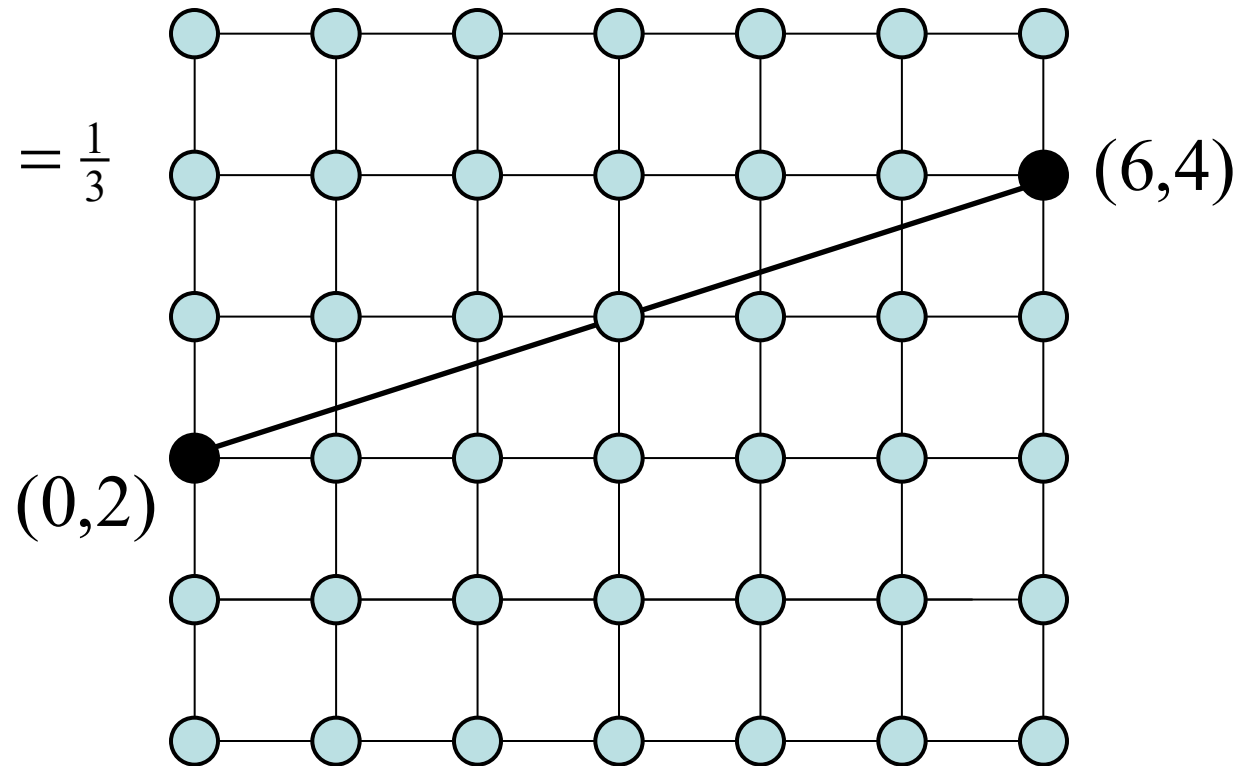


Algoritmo di Linea Incrementale: esempio

$$P_0 = (0, 2) \quad P_1 = (6, 4)$$

sarà: $n=6$, $dx=1$

$$dy = \frac{y_1 - y_0}{x_1 - x_0} = \frac{4 - 2}{6 - 0} = \frac{1}{3}$$

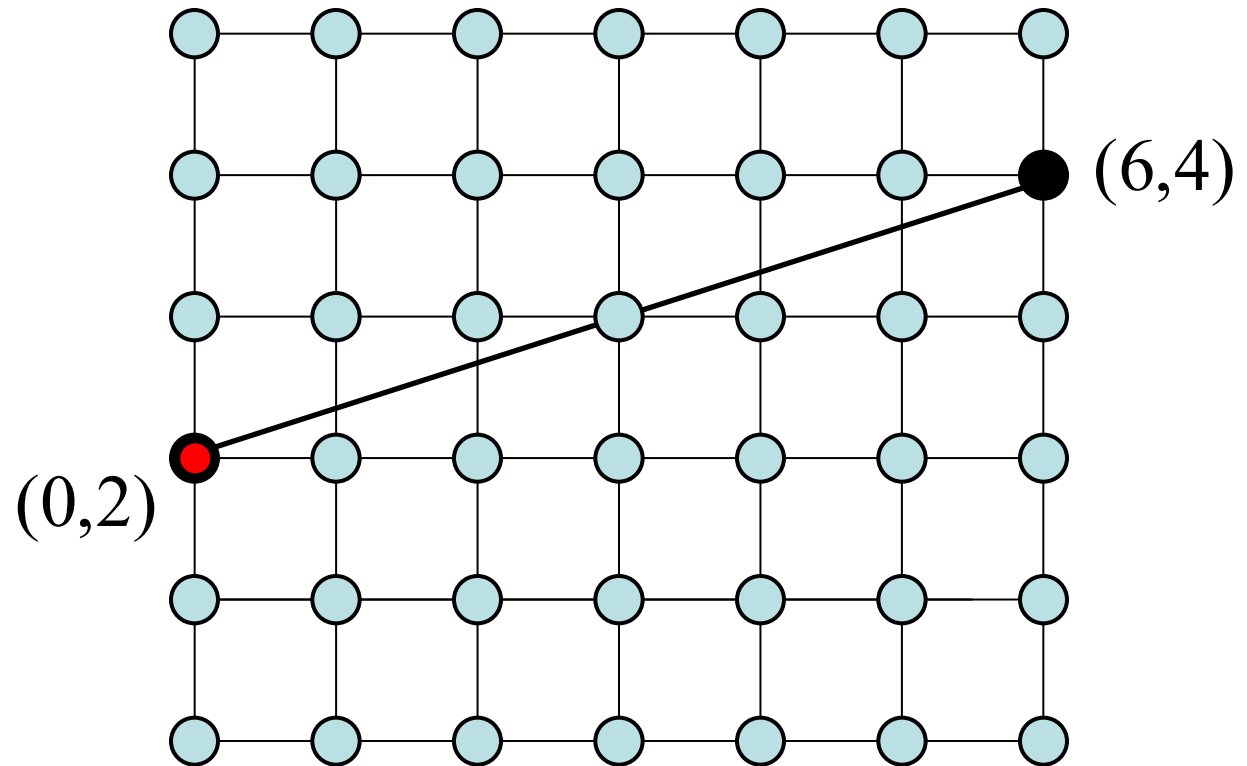




Algoritmo di Linea Incrementale: esempio

$$P_0 = (0, 2) \quad P_1 = (6, 4) \quad n=6, dx=1, dy=1/3 \cong 0.333$$

--> (0,2)

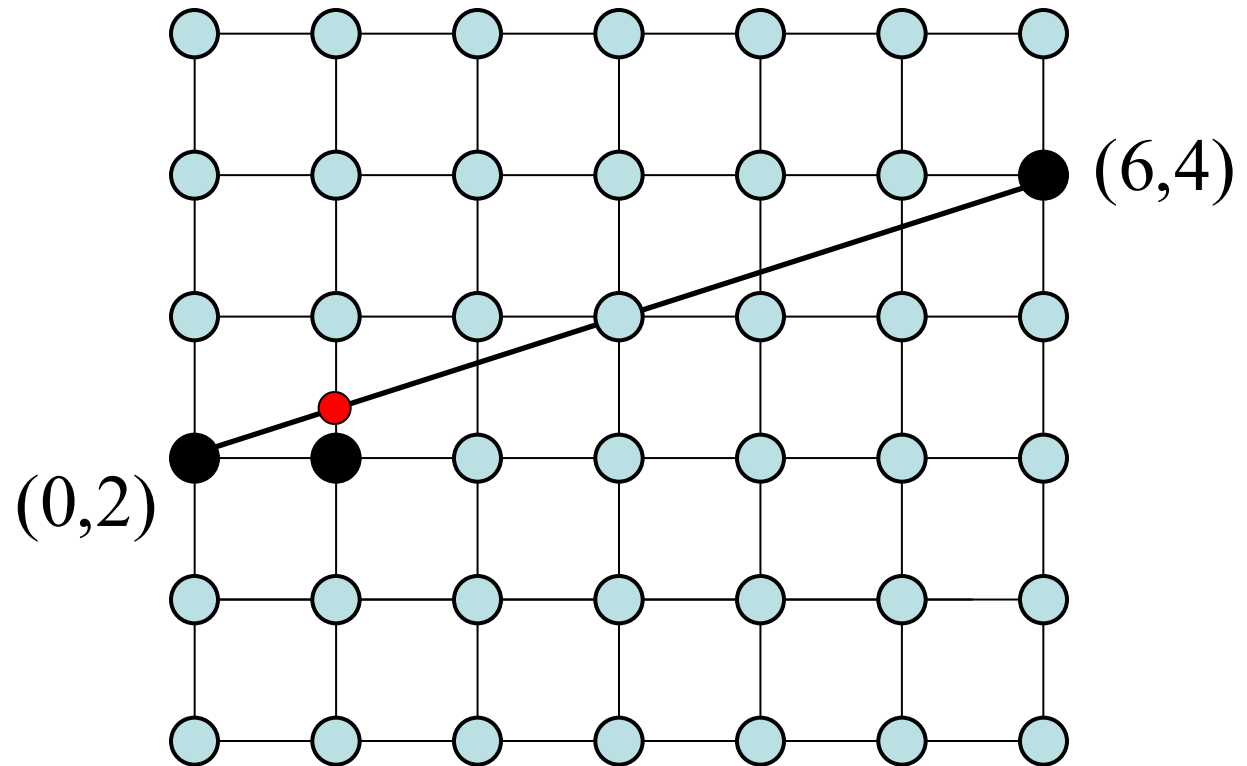




Algoritmo di Linea Incrementale: esempio

$$P_0 = (0, 2) \quad P_1 = (6, 4) \quad n=6, dx=1, dy=1/3 \cong 0.333$$

--> (0,2)
(1,2.333) --> (1,2)

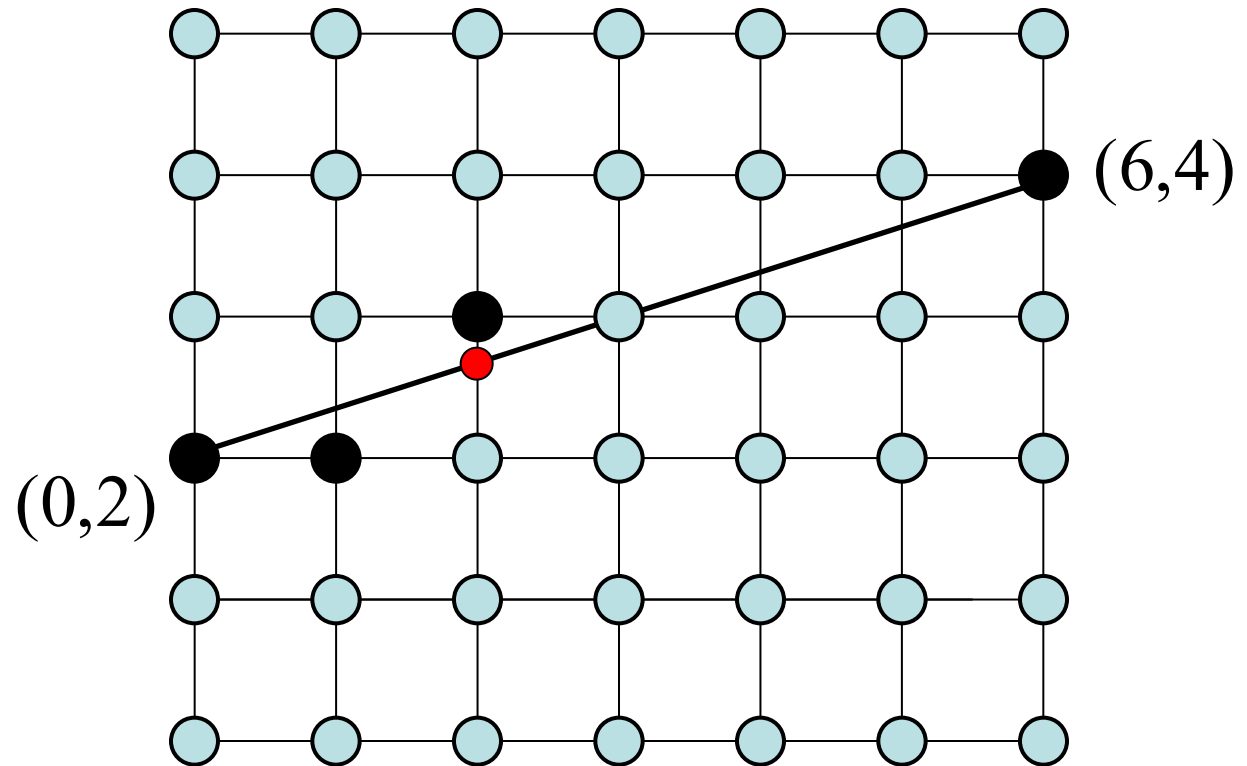




Algoritmo di Linea Incrementale: esempio

$$P_0 = (0, 2) \quad P_1 = (6, 4) \quad n=6, dx=1, dy=1/3 \simeq 0.333$$

--> (0,2)
(1,2.333) --> (1,2)
(2,2.666) --> (2,3)

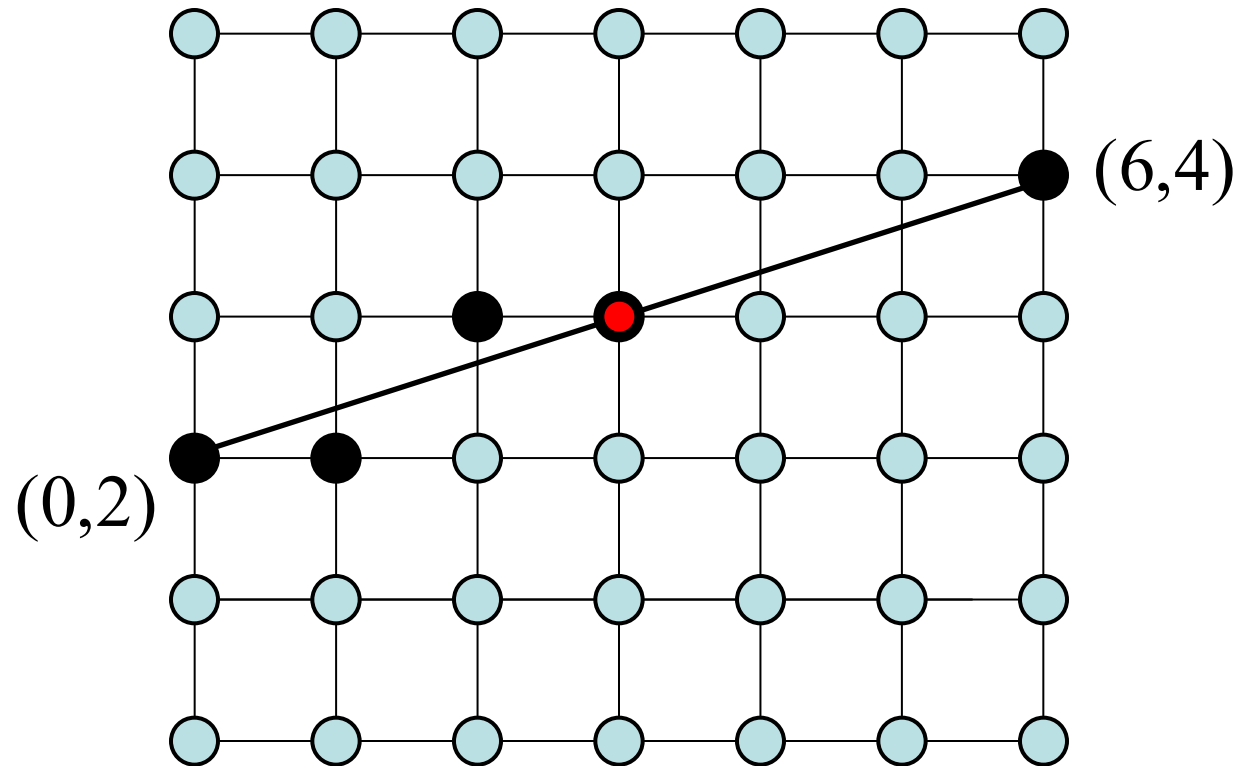




Algoritmo di Linea Incrementale: esempio

$$P_0 = (0, 2) \quad P_1 = (6, 4) \quad n=6, dx=1, dy=1/3 \simeq 0.333$$

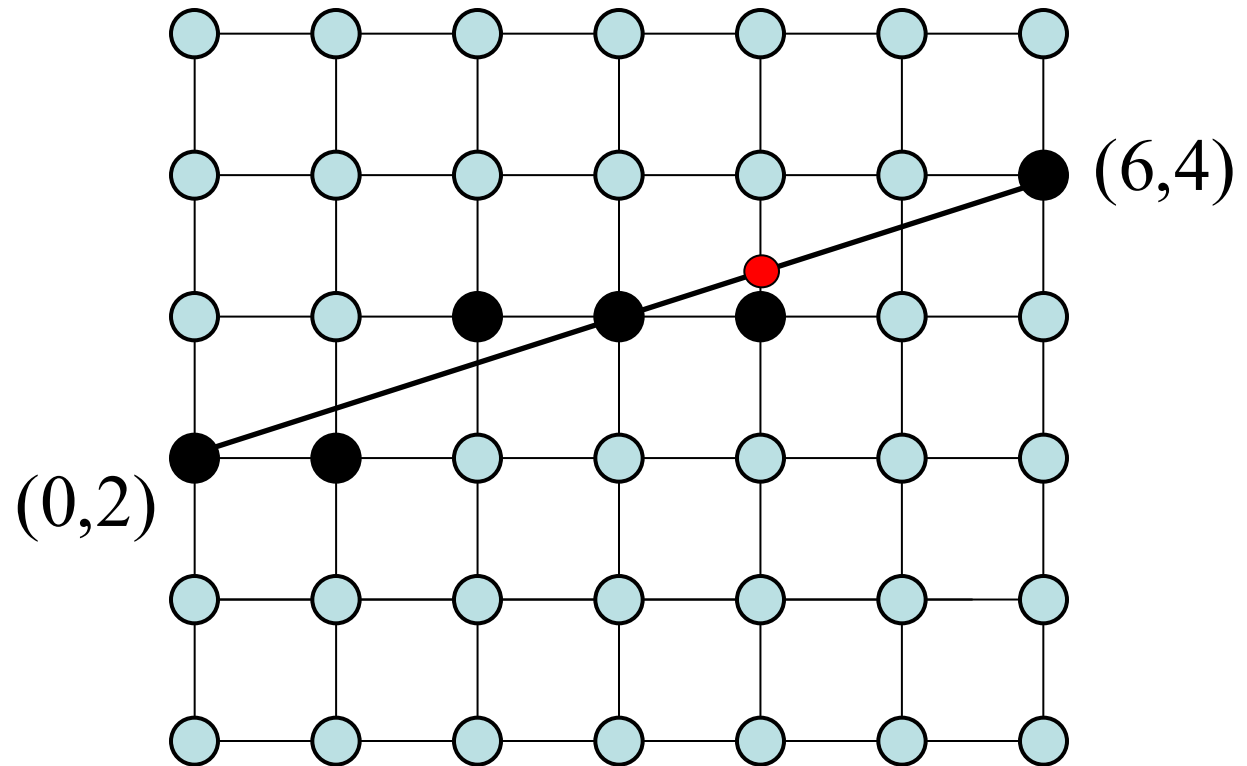
--> (0,2)
(1,2.333) --> (1,2)
(2,2.666) --> (2,3)
(3,3.000) --> (3,3)



Algoritmo di Linea Incrementale: esempio

$$P_0 = (0, 2) \quad P_1 = (6, 4) \quad n=6, dx=1, dy=1/3 \simeq 0.333$$

$--> (0,2)$
 $(1,2.333) --> (1,2)$
 $(2,2.666) --> (2,3)$
 $(3,3.000) --> (3,3)$
 $(4,3.333) --> (4,3)$

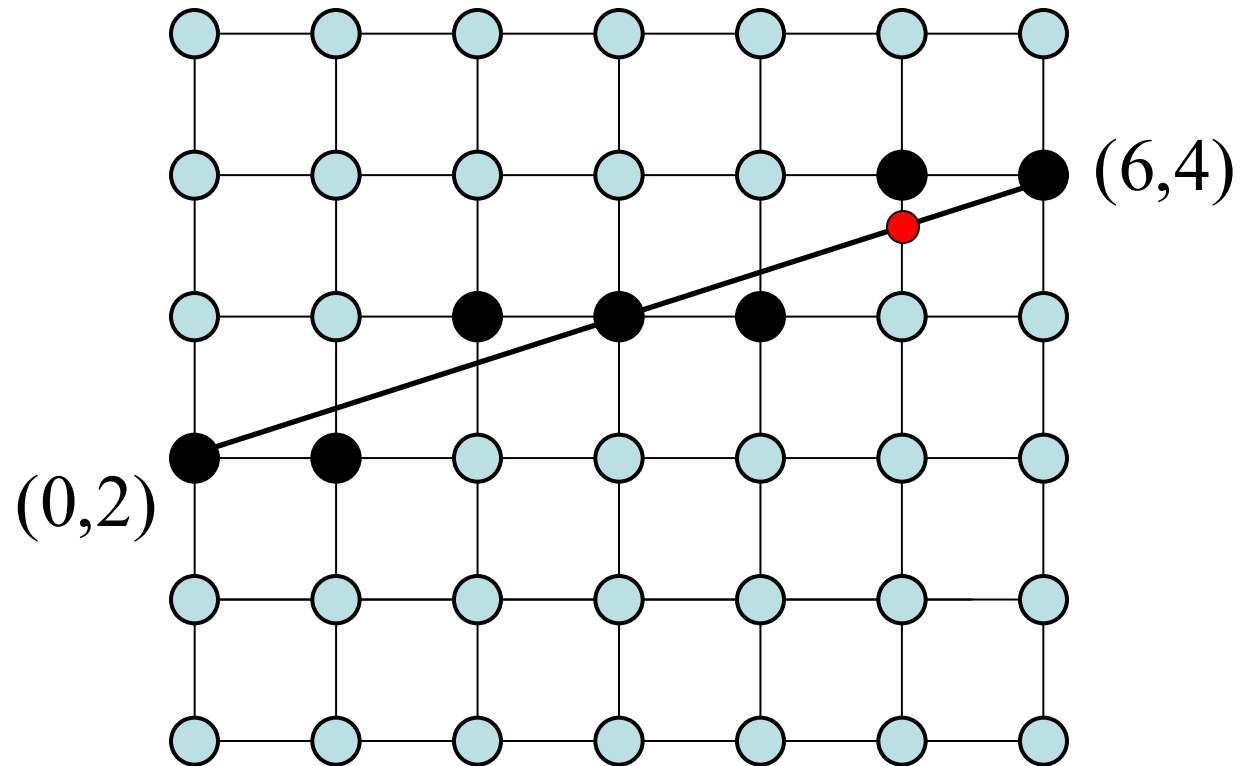




Algoritmo di Linea Incrementale: esempio

$$P_0 = (0, 2) \quad P_1 = (6, 4) \quad n=6, dx=1, dy=1/3 \simeq 0.333$$

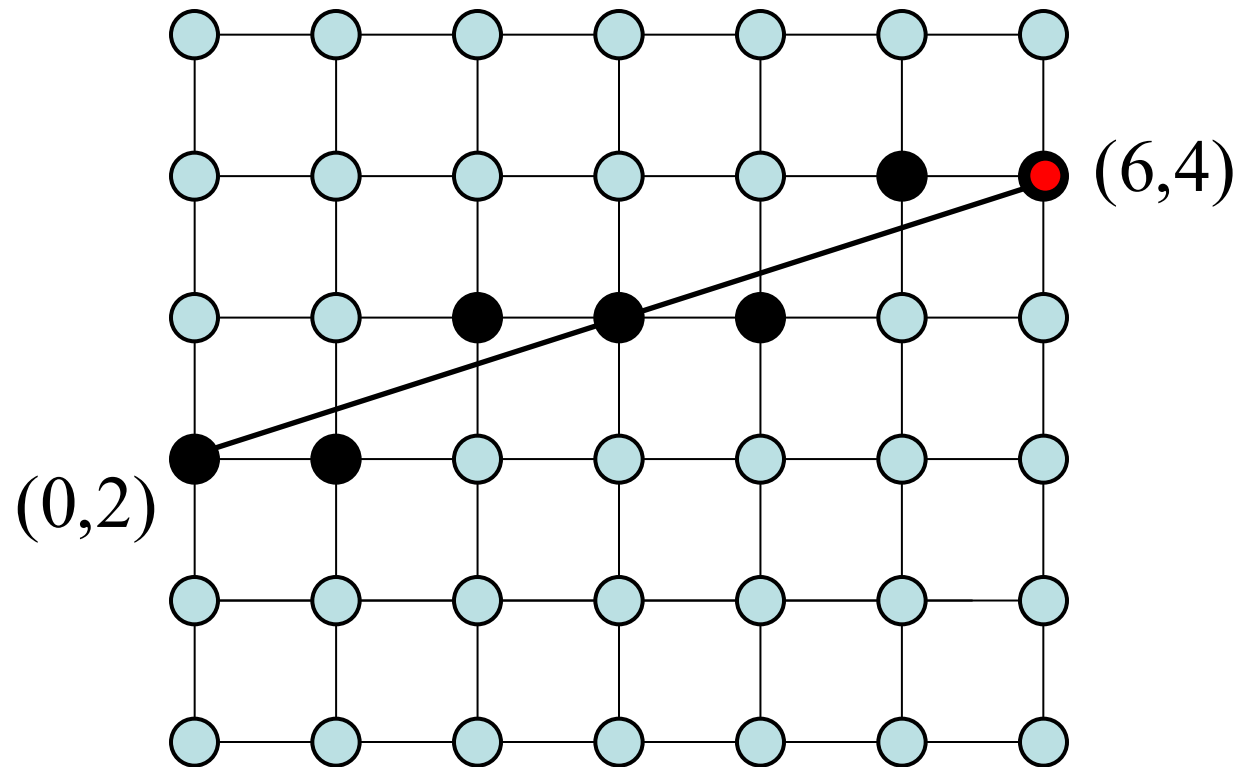
--> (0,2)
(1,2.333) --> (1,2)
(2,2.666) --> (2,3)
(3,3.000) --> (3,3)
(4,3.333) --> (4,3)
(5,3.666) --> (5,4)



Algoritmo di Linea Incrementale: esempio

$$P_0 = (0, 2) \quad P_1 = (6, 4) \quad n=6, dx=1, dy=1/3 \simeq 0.333$$

$--> (0,2)$
 $(1,2.333) --> (1,2)$
 $(2,2.666) --> (2,3)$
 $(3,3.000) --> (3,3)$
 $(4,3.333) --> (4,3)$
 $(5,3.666) --> (5,4)$
 $(6,4.000) --> (6,4)$





Algoritmo di Linea Incrementale: osservazioni

- Operazioni aritmetiche floating point
- Arrotondamento (funzione `round( )`)
- Si può fare meglio !

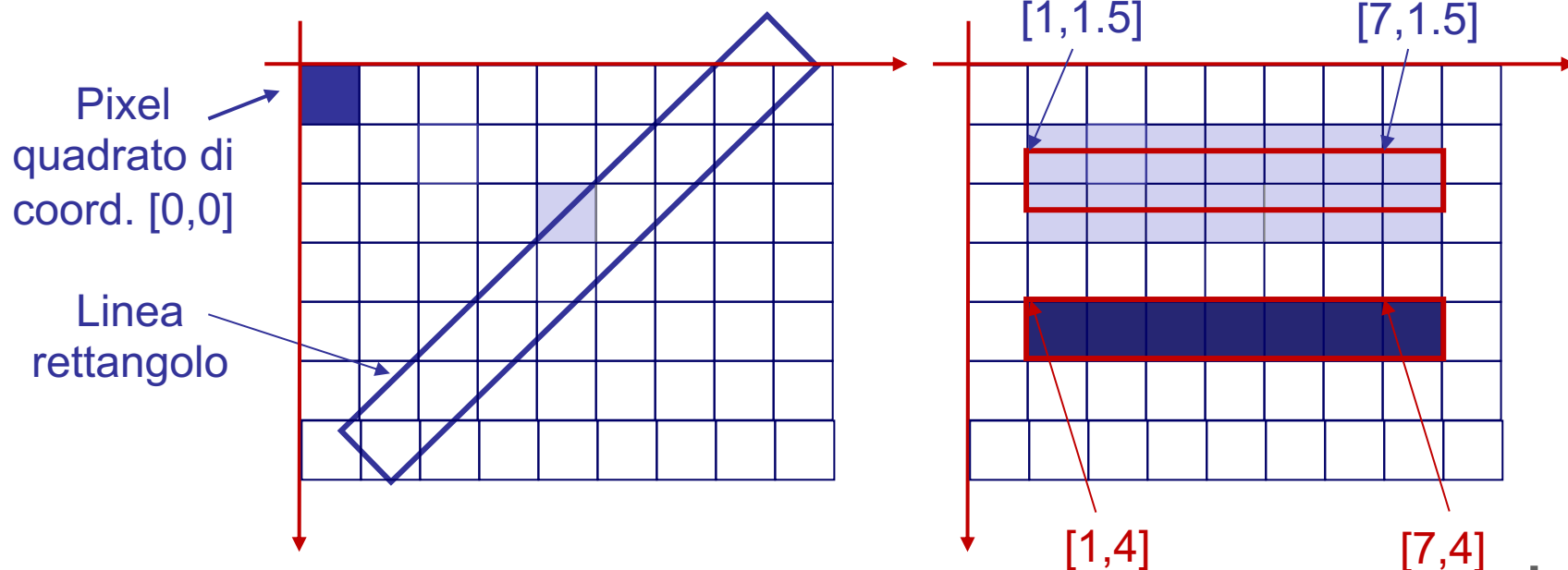


Algoritmo di Linea di Bresenham

- Solo operazioni aritmetiche fra interi
- Più precisamente addizioni, sottrazioni e shift di bit (moltiplicazioni per 2)
- Si estende ad altri tipi di forme (circonferenza, coniche)
- E' l'algoritmo implementato in hardware sulle schede grafiche
- Ne trovate una implementazione software nella cartella `HTML5_2d_1` file `draw_line.js` utilizzato dallo script `polygon_pixel.html`

Antialiasing/Smoothing

Ad ogni pixel si associa una geometria quadrato e quindi un'area; il colore dei pixel da disegnare viene determinato calcolando la percentuale di sovrapposizione del tracciato (per esempio una linea di spessore 1 pixel è rappresentata da un rettangolo) con l'area dei pixel.





Tracciati in contesto 2d

Il browser Chrome (ma anche Edge, Firefox, ecc.) utilizza **Skia**, una libreria grafica 2D open-source sviluppata da Google. Skia non usa l'algoritmo di linea di Bresenham (che è binario: acceso/spento), ma un sistema di **Analytic Rasterization**.

Ecco come Skia gestisce un tracciato (path):

- 1.**Scanline**: il tracciato viene analizzato per determinare quali pixel sono attraversati dai bordi della geometria.
- 2.**Calcolo dell'Area (Coverage)**: per ogni pixel coinvolto, Skia calcola l'integrale dell'area del pixel coperta dalla forma. Se per esempio una linea diagonale taglia un pixel a metà, il valore di copertura α sarà 0.5.
- 3.**Blending**: Questo valore di copertura viene usato come valore α per miscelare il colore. La formula semplificata è:

$$C_{\text{final}} = C_{\text{src}} \cdot \alpha + C_{\text{dst}} \cdot (1 - \alpha)$$

dove C_{src} è il colore della linea e C_{dst} è il colore già presente sul canvas.



Problema: disegno in coord. float

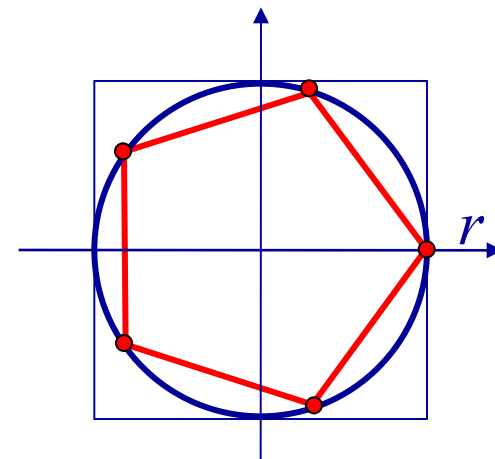
Disegnare un poligono regolare di n lati.

Dati: n numero di lati (o vertici)

r raggio circonferenza circoscritta

Risultato: disegno a pixel del poligono

Metodo: si usa l'equazione parametrica della circonferenza di centro l'origine e raggio unitario e si determinano n punti equidistanti su di essa.



Algoritmo:

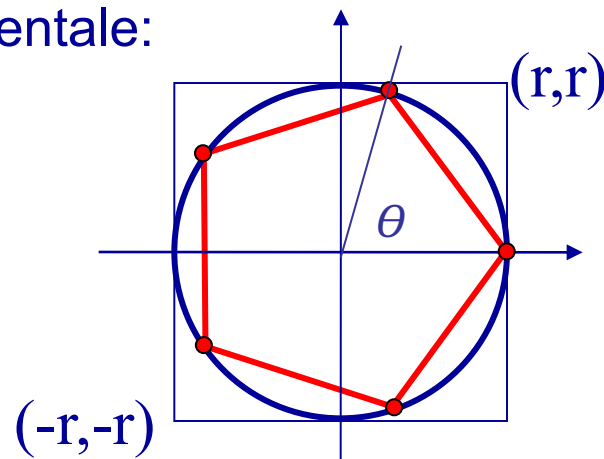
```
theta = 6.28/n
for (i = 0; i <= n; i++)
{
    t = i * theta
    x[i] = r * cos( t )
    y[i] = r * sin( t )
}
```

$$\begin{cases} x = \cos (t) \\ y = \sin (t) \\ t \in [0, 2\pi[\end{cases}$$

Problema: disegno in coord. float

Per ottimizzare, si può usare un metodo incrementale:

```
theta = 6.28/n
c=cos(theta)
s=sin(theta)
x[0] = r
y[0] = 0
for (i =1; i<=n; i++)
{
    x[ i ] = x[i-1] * c - y[i-1] * s
    y[ i ] = x[i-1] * s + y[i-1] * c
}
```



$$\begin{bmatrix} p'_x \\ p'_y \end{bmatrix} = \begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix} \cdot \begin{bmatrix} p_x \\ p_y \end{bmatrix}$$

Ma i punti così determinati sono in coordinate floating point e in un quadrato $[-r, r] \times [-r, r]$ che chiameremo **Window**; Dobbiamo disegnare su una **Viewport** sullo schermo in coordinate intere.

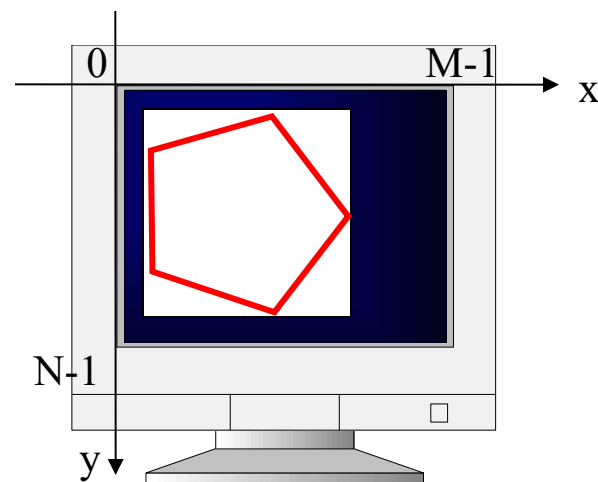
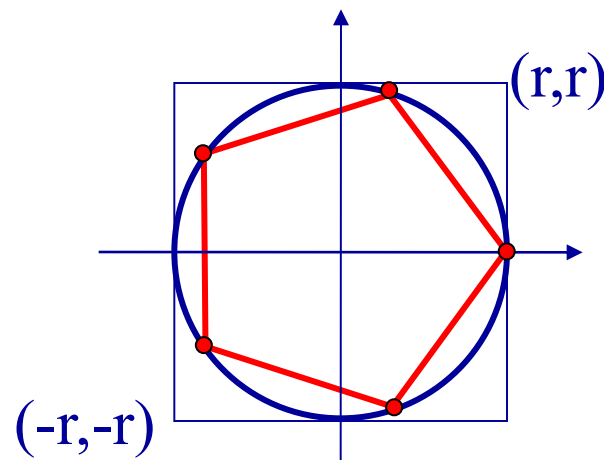
Si definisca una **Viewport** con lo stesso **aspect ratio** (rapporto fra i lati) della **Window**.

Definizioni di Window e Viewport

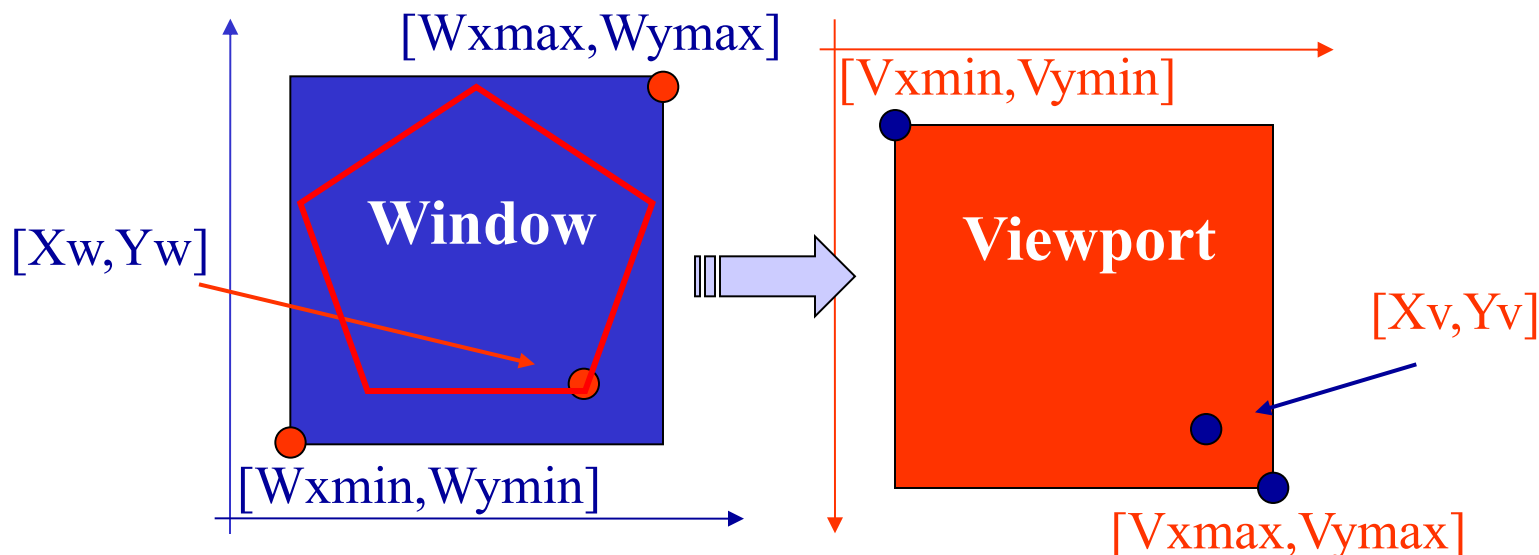
Chiameremo **Window** un'area rettangolare del piano di disegno che tipicamente contiene il nostro disegno.

Chiameremo **Viewport** un'area rettangolare dello schermo su cui vogliamo rappresentare il disegno.

Problema: dovremo trasformare le coordinate floating point (**Window**) in coordinate intere (**Viewport**)



Trasformazione Window-Viewport



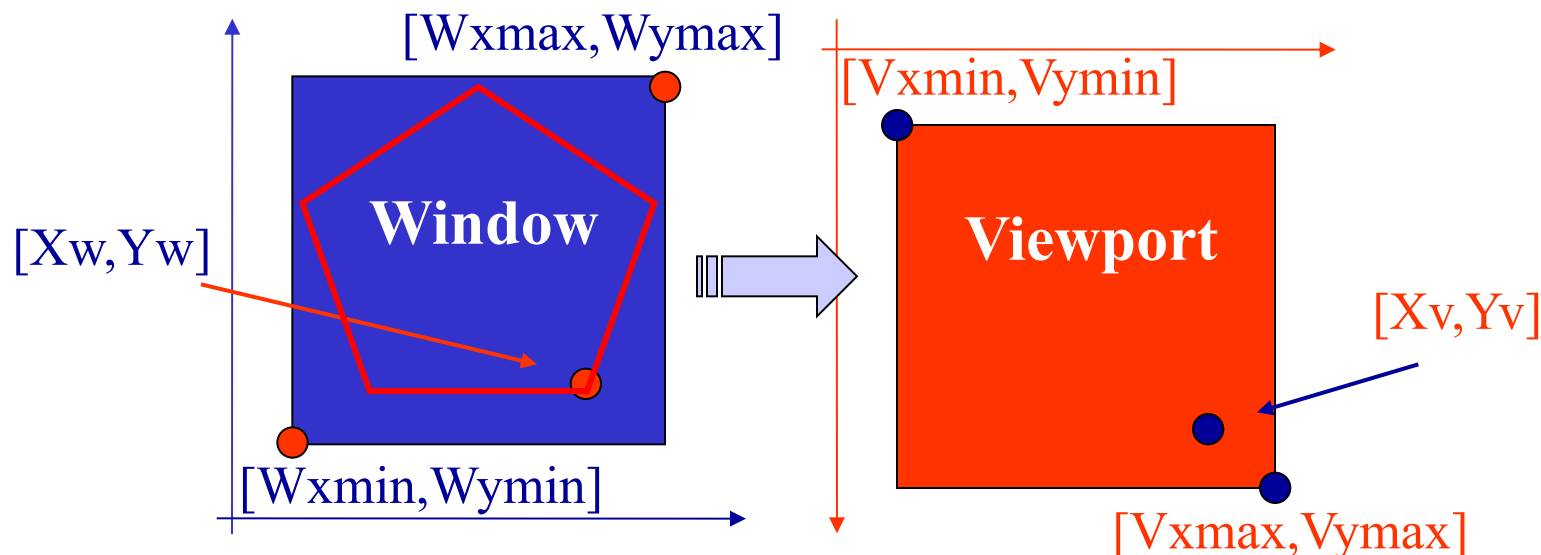
Gestiamo separatamente i due assi coordinati:

$$(X_v - V_{xmin}) : (V_{xmax} - V_{xmin}) = (X_w - W_{xmin}) : (W_{xmax} - W_{xmin})$$

da cui: $X_v = S_{cx} (X_w - W_{xmin}) + V_{xmin}$

dove $S_{cx} = (V_{xmax} - V_{xmin}) / (W_{xmax} - W_{xmin})$

Trasformazione Window-Viewport



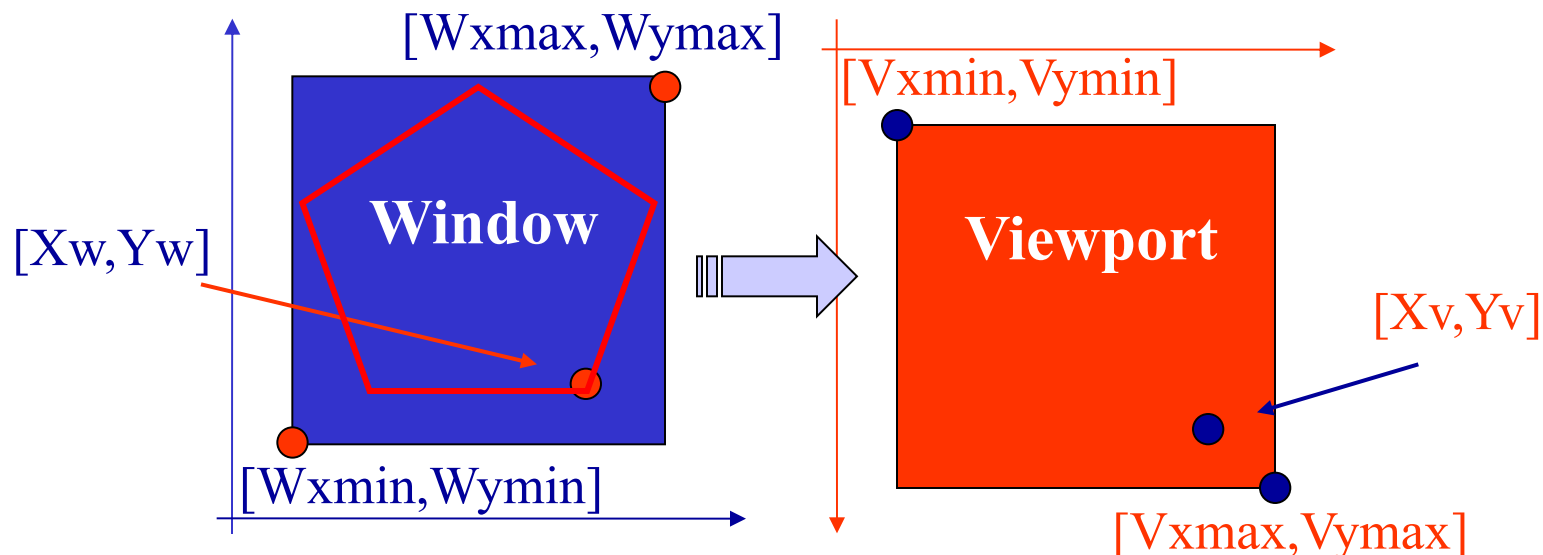
Analogamente per l'asse coordinato Y :

$$(V_{ymax} - Y_v) : (V_{ymax} - V_{ymin}) = (y_w - W_{ymin}) : (W_{ymax} - W_{ymin})$$

da cui: $Y_v = S_{cy} (W_{ymin} - Y_w) + V_{ymax}$

dove $S_{cy} = (V_{ymax} - V_{ymin}) / (W_{ymax} - W_{ymin})$

Trasformazione Window-Viewport



Che possiamo implementare così:

$$X_v = (\text{int})(Sc_x * (X_w - W_{xmin}) + V_{xmin} + 0.5)$$

$$Y_v = (\text{int})(Sc_y * (W_{ymin} - Y_w) + V_{ymax} + 0.5)$$

con

$$Sc_x = (V_{xmax} - V_{xmin}) / (W_{xmax} - W_{xmin})$$

$$Sc_y = (V_{ymax} - V_{ymin}) / (W_{ymax} - W_{ymin})$$

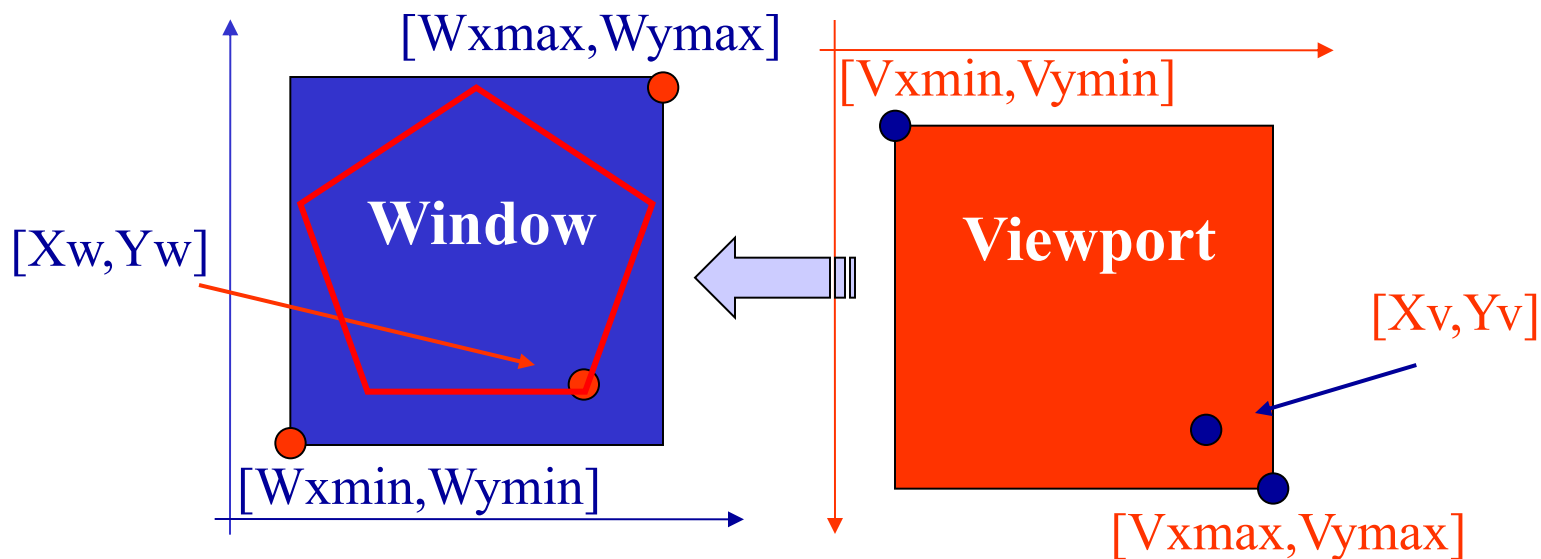


Trasformazione Window-Viewport

Disegnare in coordinate floating point ha notevoli vantaggi:

- la viewport può essere posizionata in diversi punti dello schermo, avere differenti dimensioni e aspect ratio, ma non sarà necessario cambiare il disegno, bensì basterà riapplicare il mapping window-viewport;
- zoom: ridefinire la window rispetto a quanto disegnato e applicare il mapping window-viewport permette di vedere a pieno schermo dettagli di quanto disegnato;
- la trasformazione inversa viewport-window permette di definire interattivamente le parti del disegno da zoomare;
- ecc.

Trasformazione Inversa Viewport-Window



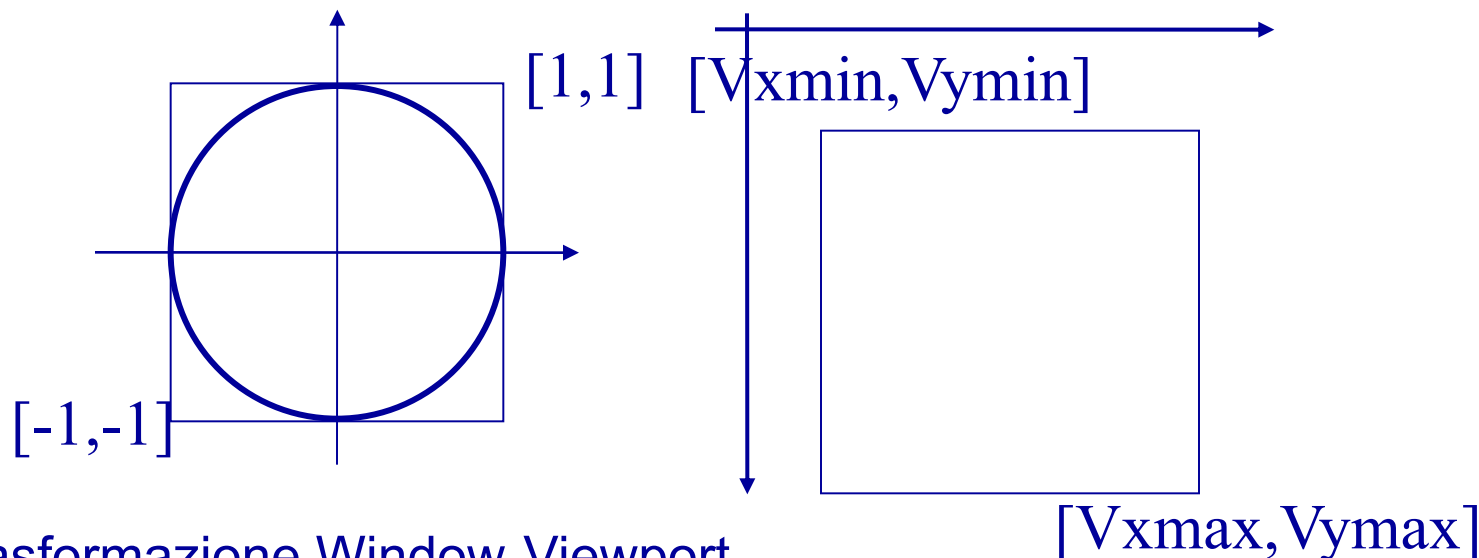
Trasformazione inversa:

$$X_w = (X_v - V_{xmin}) / S_{cx} + W_{xmin}$$

$$Y_w = (V_{ymax} - Y_v) / S_{cy} + W_{ymin}$$



Problema: disegno in coord. float



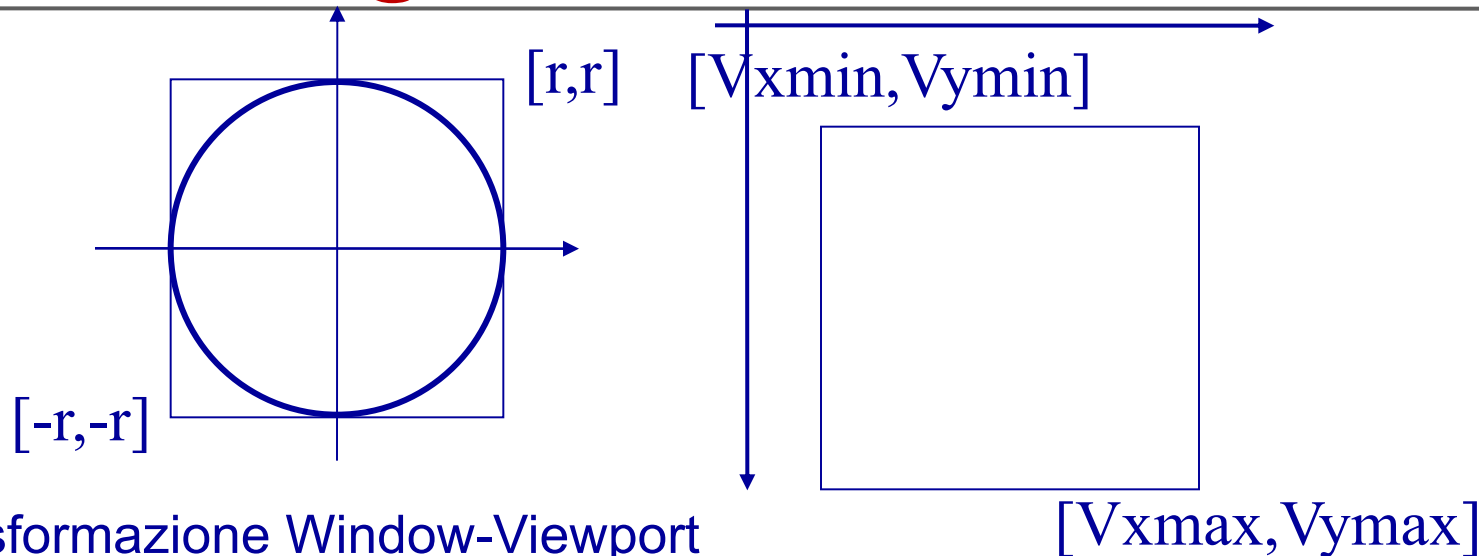
Trasformazione Window-Viewport
Definiamo due strutture/oggetti:

```
var view={  
  xmin: 0;  
  xmax: 300;  
  ymin: 0;  
  ymax: 150;  
}
```

```
var win={  
  xmin: -1.0;  
  xmax: 1.0;  
  ymin: -1.0;  
  ymax: 1.0;  
}
```



Problema: disegno in coord. float



Trasformazione Window-Viewport

Scriviamo una funzione

```
function win_view(px,py,scx,scy,view,win)
```

Si applichi la trasformazione suddetta ai punti $(x[i], y[i])$ $i=0, \dots, n$ e si disegni su schermo la spezzata di vertici così ottenuti.

vedi codice: [polygon_float.html](#)



Esercizio

Avendo visto il codice che disegna i punti floating point di un poligono discretizzando una circonferenza su una Viewport ([HTML5_2d_1/polygon_float.js](#)), realizzare un codice per il disegno di una curva piana definita in forma parametrica:

$$C(t) = [C_x(t), C_y(t)] \quad t \in [0,1]$$

lo script si chiami [draw_param_curve.html \(.js\)](#)

Per esempi di curve in forma parametrica si veda:

<http://mathshistory.st-andrews.ac.uk/Curves/Curves.html>



Soluzione

Problema: produrre il grafico di una curva in forma parametrica.

Dati: espressione $c(t) = [f(t), g(t)]$
con $t \in [a, b]$ intervallo di definizione

Risultato: grafico di $n+1$ punti della curva

Metodo: si campiona la curva in $n+1$ punti, per esempio equispaziati nell'intervallo parametrico di definizione

Algoritmo: tabulazione della curva

```
h = (b-a)/n;  
for (var i = 0; i <= n; i++){  
    t = a + i*h;  
    x[i] = f(t)  
    y[i] = g(t)  
}
```



Soluzione

Bisogna determinare la Window, cioè il più piccolo rettangolo che contiene i punti $(x[i], y[i])$, $i=0, \dots, n$

```
win.xmin = x[0]
win.xmax = x[0]
for (var i=0; i<n; i++)
{
    if (x[i]>win.xmax)
        win.xmax=x[i]
    else
        if (x[i]<win.xmin)
            win.xmin=x[i]
}
```

```
win.ymin = y[0]
win.ymax = y[0]
for (var i=0; i<n; i++)
{
    if (y[i]>win.ymax)
        win.ymax=y[i]
    else
        if (y[i]<win.ymin)
            win.ymin=y[i]
}
```

La Window sarà:

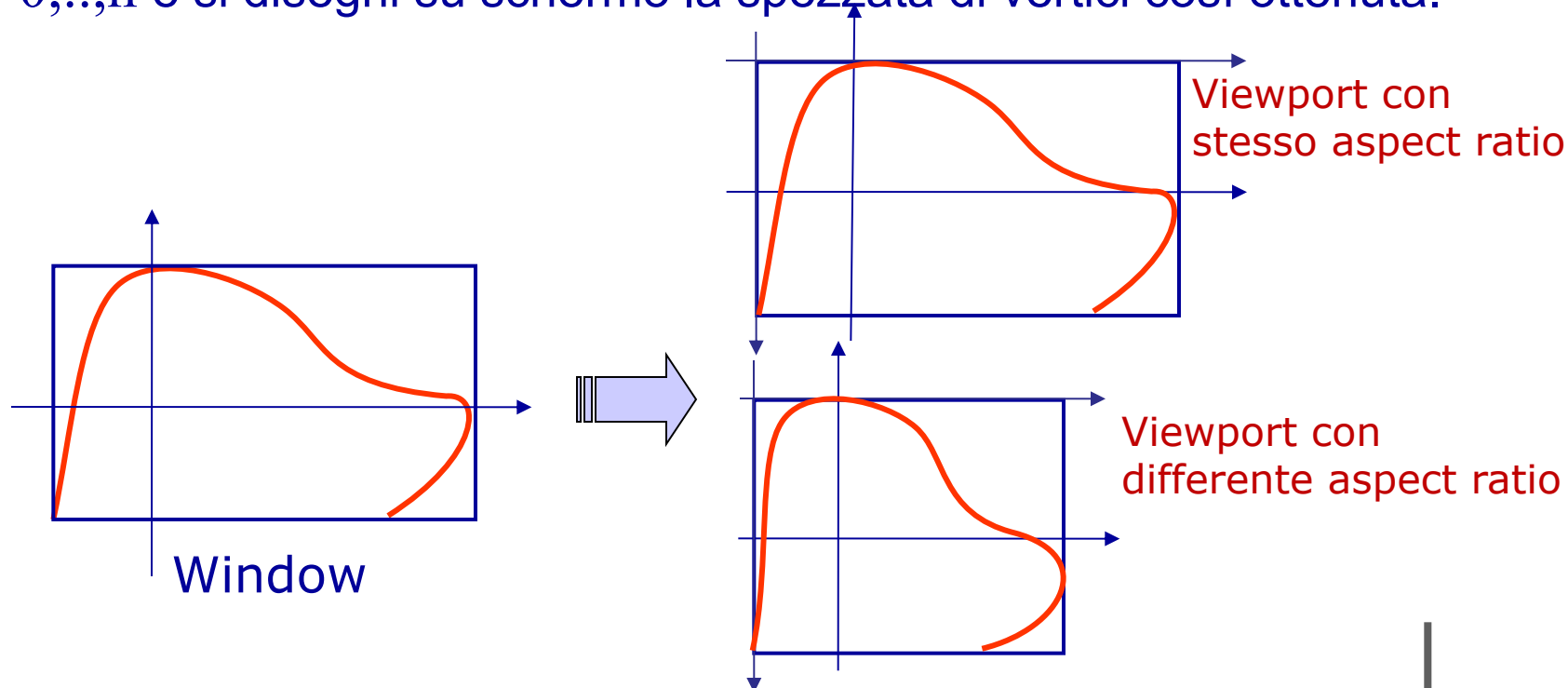
$[win.xmin, win.xmax] \times [win.ymin, win.ymax]$

Soluzione

Si definisca una Viewport con lo stesso aspect ratio (rapporto fra i lati) della Window; definito quindi:

$[view.xmin, view.xmax] \times [view.ymin, view.ymax]$

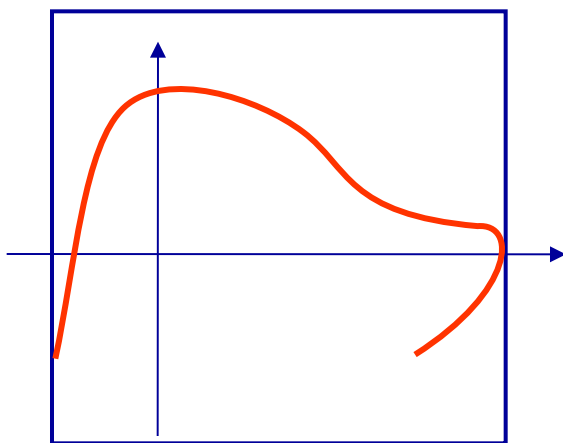
si applichi la trasformazione Window-Viewport ai punti $(x[i], y[i])$ $i=0, \dots, n$ e si disegni su schermo la spezzata di vertici così ottenuta.



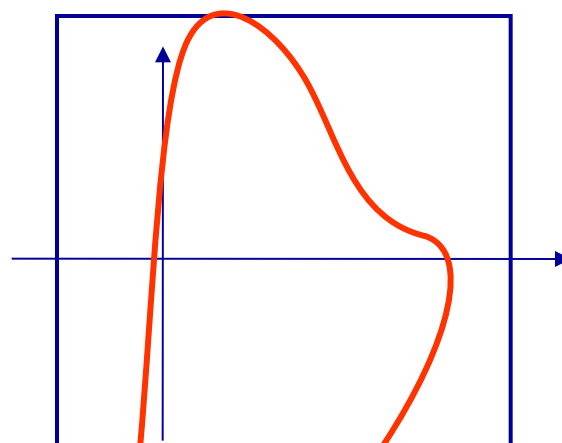
Soluzione

In alternativa si definisca una Viewport quadrata e si determini la più piccola Window quadrata contenente i punti $(x[i], y[i])$, $i=0, \dots, n$, così da avere una rappresentazione corretta delle proporzioni.

Si applichi poi la trasformazione suddetta ai punti $(x[i], y[i])$ $i=0, \dots, n$ e si disegni su schermo la spezzata di vertici così ottenuti.



Window



Window

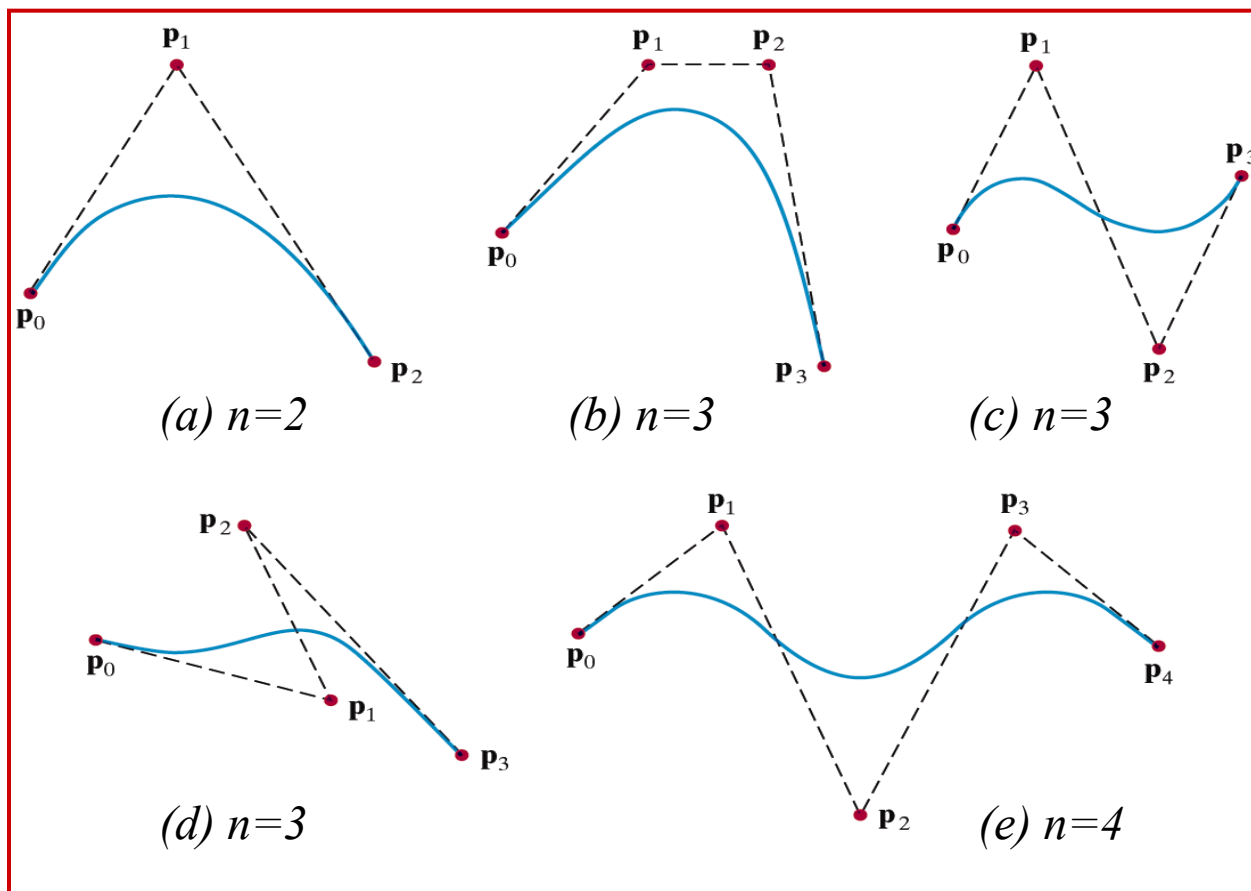


Soluzione

```
dx=win.xmax-win.xmin;  
dy=win.ymax-win.ymin;  
if (dy > dx){  
    diff=(dy-dx)/2;  
    win.xmin=win.xmin-diff;  
    win.xmax=win.xmax+diff;  
}  
else {  
    diff=(dx-dy)/2;  
    win.ymin=win.ymin-diff;  
    win.ymax=win.ymax+diff;  
}
```

Curve di Bézier

Esempi di curve di Bézier di grado $n=2,3,4,\dots$





Codice di Esempio

Si analizzi il codice `HTML5_2d_1/bezier.html + .js` che permette di definire e disegnare interattivamente una poligonale di controllo di 4 vertici (coordinate intere schermo), quindi disegna la curva di Bézier di grado 3 che ne resta definita.

Una volta disegnata la curva il codice permette di modificarne la forma spostando col mouse singoli punti di controllo;

Per valutare la curva viene utilizzata la function `bezierCurveTo` presente nell'API del contesto 2d.



Esercizio 1

Realizzare un codice che permetta di definire e disegnare interattivamente una poligonale di controllo di $n+1$ vertici (coordinate floating point window), quindi disegni la curva di Bézier di grado n , che ne resta definita (per valutare la curva si usi l'**algoritmo di valutazione di de Casteljau**).

Una volta disegnata la curva sia possibile modificarne la forma spostando col mouse singoli punti di controllo;

$$C(t) = \sum_{i=0}^n P_i B_{i,n}(t) \quad t \in [0,1]$$

Punti di controllo

base di Bernstein

Lo script si chiama **draw_bezier_curve.html + .js**



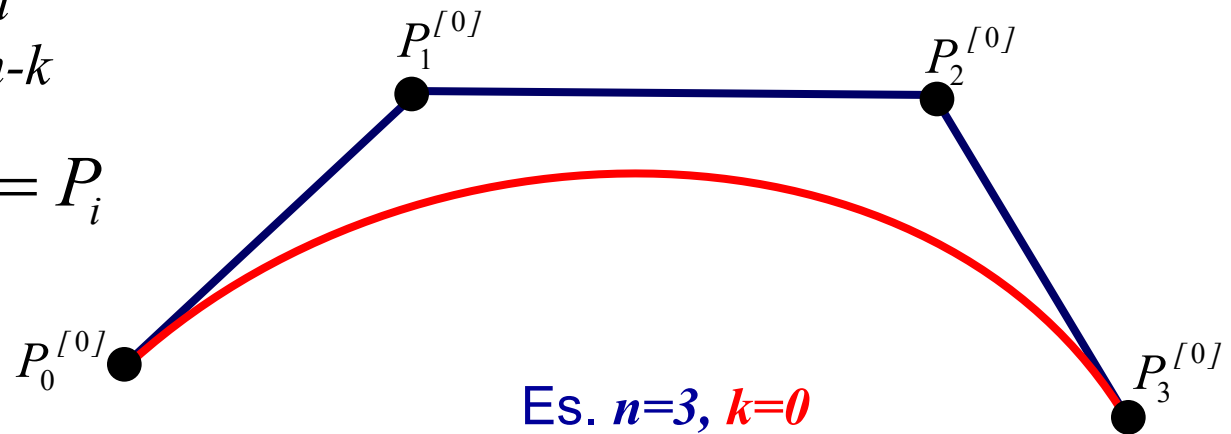
Curve di Bézier e algoritmo di valutazione di de Casteljau

Il matematico francese de Casteljau, negli anni '60, diede una definizione di curva di Bézier basata su “corner cutting” successivi:

$$P_i^{[k]}(t) = (1-t)P_i^{[k-1]}(t) + tP_{i+1}^{[k-1]}(t) \quad t \in [0,1]$$

dove $k = 1, \dots, n$
 $i = 0, \dots, n-k$

con $P_i^{[0]}(t) = P_i$
 $i = 0, \dots, n$



Nota: i P_i sono i punti di controllo di definizione della curva di Bézier



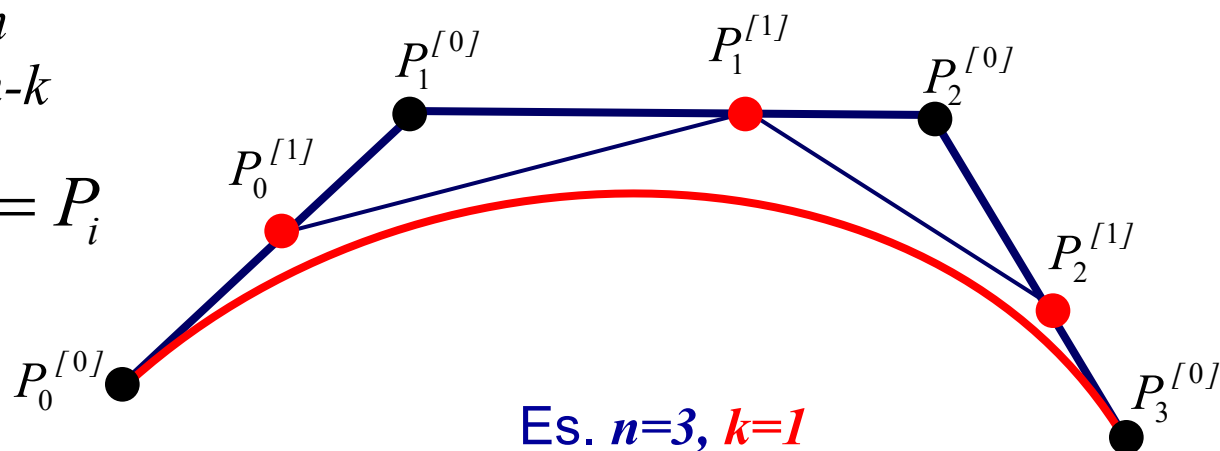
Curve di Bézier e algoritmo di valutazione di de Casteljau

Il matematico francese de Casteljau, negli anni '60, diede una definizione di curva di Bézier basata su “corner cutting” successivi:

$$P_i^{[k]}(t) = (1-t)P_i^{[k-1]}(t) + tP_{i+1}^{[k-1]}(t) \quad t \in [0,1]$$

dove $k = 1, \dots, n$
 $i = 0, \dots, n-k$

con $P_i^{[0]}(t) = P_i$
 $i = 0, \dots, n$



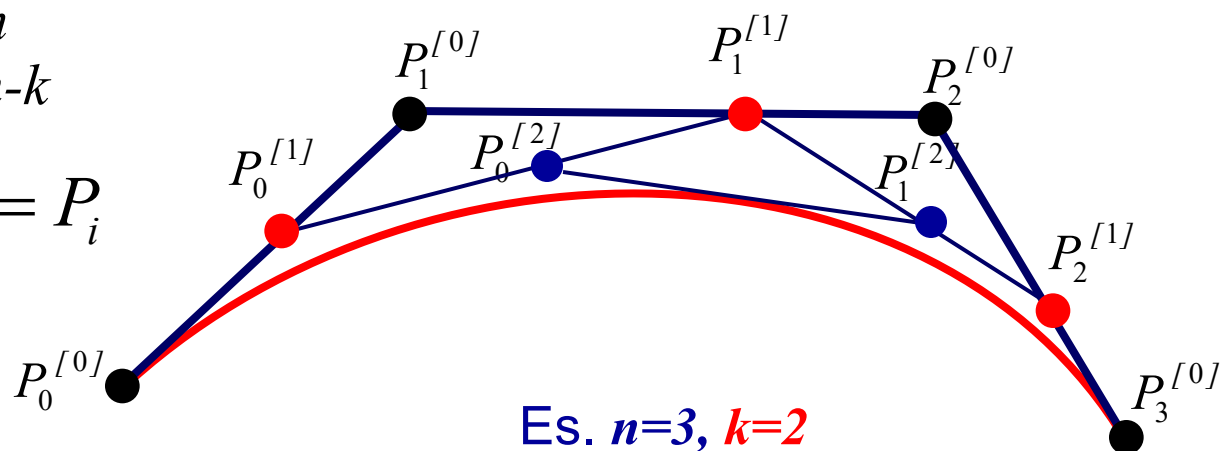
Curve di Bézier e algoritmo di valutazione di de Casteljau

Il matematico francese de Casteljau, negli anni '60, diede una definizione di curva di Bézier basata su “corner cutting” successivi:

$$P_i^{[k]}(t) = (1-t)P_i^{[k-1]}(t) + tP_{i+1}^{[k-1]}(t) \quad t \in [0,1]$$

dove $k = 1, \dots, n$
 $i = 0, \dots, n-k$

con $P_i^{[0]}(t) = P_i$
 $i = 0, \dots, n$





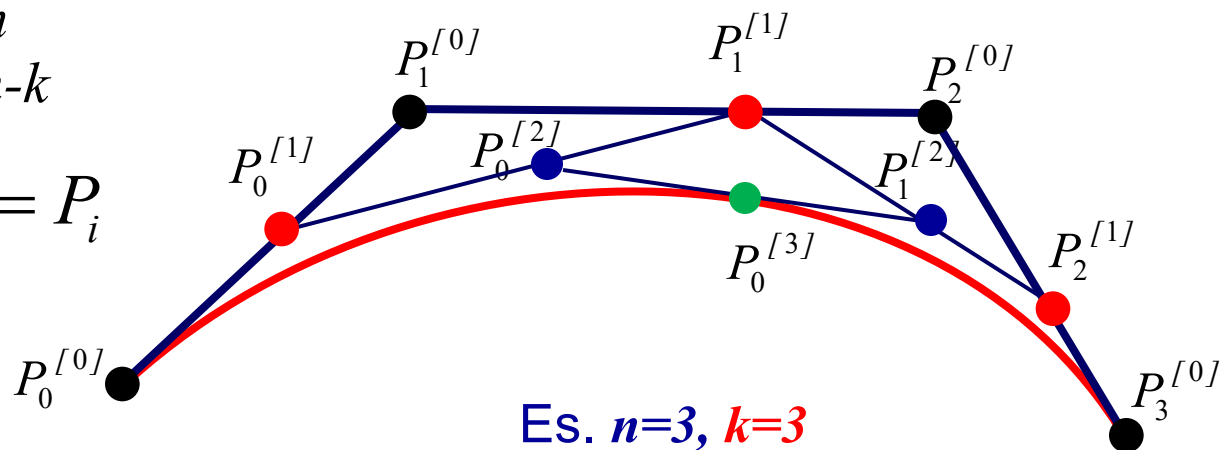
Curve di Bézier e algoritmo di valutazione di de Casteljau

Il matematico francese de Casteljau, negli anni '60, diede una definizione di curva di Bézier basata su “corner cutting” successivi:

$$P_i^{[k]}(t) = (1-t)P_i^{[k-1]}(t) + tP_{i+1}^{[k-1]}(t) \quad t \in [0,1]$$

dove $k = 1, \dots, n$
 $i = 0, \dots, n-k$

con $P_i^{[0]}(t) = P_i$
 $i = 0, \dots, n$



Questa definizione è anche un algoritmo numericamente stabile per il calcolo delle curve di Bézier.



Codice Alg. de Casteljau

```
//Algoritmo di de Casteljau per valutazione curva di Bézier
//di grado n in coordinate floating point

function compute_bezier(){

// input
// fxt, fyt: array di ascisse e ordinate dei punti di controllo
// n: grado della curva
// m: numero punti di valutazione - 1 (m+1)
// output
// vxp, vyp: array di ascisse e ordinate dei punti della curva

var h=1/m; //passo punti di valutazione
var fxp=[], fyp=[]; //array di lavoro
var t,d1,d2;

//primo punto della curva
vxp[0]=fxt[0];
vyp[0]=fyt[0];
```

Codice: `compute_bezier.js`



Codice Alg. de Casteljau

```
//calcola gli m-1 punti interni
for (var k=1; k<m; k++){
    fxp=Array.from(fxt);
    fyp=Array.from(fyt);
    t=k*h;
    d1=t;
    d2=1.0-t;
    for (var j=1; j<n; j++)
        for (var i=0; i<n-j; i++){
            fxp[i]=d1*fxp[i+1]+d2*fxp[i];
            fyp[i]=d1*fyp[i+1]+d2*fyp[i];
        }
    vxp[k]=fxp[0];
    vyp[k]=fyp[0];
}
//ultimo punto della curva
vxp[m]=fxt[n];
vyp[m]=fyt[n];
win_view_convert(); //da coordinate floating point (window) a
                    //coordinate intere (viewport)
}
```

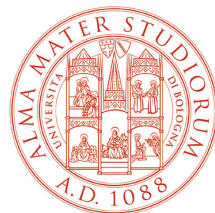


Esercizio 1 (richieste opzionali)

Si modifichi il codice realizzato per l'Esercizio 1 in modo che:

1. sia possibile applicare **trasformazioni geometriche** elementari alla curva di Bézier;
2. sia possibile effettuare un **panning** (spostare la curva nell'area rettangolare viewport)
3. sia possibile effettuare uno **zooming to fit** della curva alla viewport (ingrandire la curva a piena viewport)
4. sia possibile effettuare lo **zoom** di un particolare

Il codice si chiami **draw_bezier_curve2.html + .js**



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Giulio Casciola
Dip. di Matematica
giulio.casciola@unibo.it